

VIETNAM NATIONAL UNIVERSITY, HANOI

VIETNAM JAPAN UNIVERSITY

FALCULTY OF ADVANCED TECHNOLOGIES AND ENGINEERING

Nguyen Huu Giap

**DESIGN AND IMPLEMENTATION OF A SMART
AGRICULTURE MONITORING AND CONTROL
SYSTEM USING LORA NETWORK INTEGRATED
WITH AI-BASED ENVIROMENTAL FORECASTING**

Bachelor in Computer Science and Engineering

(Honors Program)

Hanoi – 2026

VIETNAM NATIONAL UNIVERSITY, HANOI

VIETNAM JAPAN UNIVERSITY

FALCULTY OF ADVANCED TECHNOLOGIES AND ENGINEERING

Nguyen Huu Giap

**DESIGN AND IMPLEMENTATION OF A SMART
AGRICULTURE MONITORING AND CONTROL
SYSTEM USING LORA NETWORK INTEGRATED
WITH AI-BASED ENVIROMENTAL FORECASTING**

Bachelor in Computer Science and Engineering

(Honors Program)

Supervisor(s):

Ph.D. Tran Huu Phi

Ph.D. Bui Huy Kien

Hanoi – 2026

DECLARATION

I hereby commit that the entire content of this report/thesis is the result of my own learning, research, and implementation process under the guidance of my supervising instructors. All data, results, and reference materials presented in this report are clearly cited, truthfully and fully compliant with academic ethics regulations.

I take full responsibility before the university and the law for any copying, fraud, or copyright infringement in this report.

Ha Noi, May 20, 2026

Student

ACKNOWLEDGMENT

To complete this research project, I would like to express my deep gratitude to the Board of Directors of the Faculty of Advanced Technology and Engineering, Vietnam Japan University - Vietnam National University, Hanoi, especially to the lecturers who have directly taught, enthusiastically helped and provided valuable contributions during the learning and research process.

I would like to send my thanks to Dr. Tran Huu Phi - Lecturer at the Military Technical Academy, Hanoi, who has directly guided, instructed me wholeheartedly, and always encouraged and helped me greatly during my study period, research implementation, and completion of this project.

I also gratefully acknowledge Dr. Bui Huy Kien, who offered insightful suggestions, valuable feedback, and actively contributed to shaping and refining the content of this research, helping me improve the quality and effectiveness of the final outcome.

I want to thank my relatives and friends who have encouraged and helped me in all aspects so that I could successfully complete this scientific research project.

Due to time constraints and limited capabilities, the content of this research cannot avoid shortcomings and deficiencies. I sincerely hope to receive feedback from the university, lecturers, and colleagues so that I can have the opportunity to correct, supplement, and improve my research.

I sincerely thank you!

Ha Noi, May 20, 2026

Student

TABLE OF CONTENTS

PREFACE	1
CHAPTER 1: THESIS OVERVIEW	2
1.1. Research necessity	2
1.2. Problem statement	2
1.3. Literature review	3
1.4. Research objectives	4
1.5. Significance of the thesis	4
CHAPTER 2: SYSTEM ARCHITECTURE	5
2.1. Overall System Architecture.....	5
2.2. Detailed system architecture diagram.....	6
2.2.1. Field model block.....	6
2.2.2. Central coordination gateway block (Master Node)	7
2.2.3. Intermediate cloud Infrastructure block (Firebase Cloud)	7
2.2.4. Application and AI layer	7
2.3. Data flow and control mechanism	8
2.3.1. Monitoring data flow (Uplink)	8
2.3.2. Control command flow (Downlink).....	8
2.4. Communication Network Structure	8
CHAPTER 3: TECHNOLOGICAL FOUNDATIONS	9
3.1. LoRa communication technology.....	9
3.1.1. Introduction to LoRa technology	9
3.1.2. Operational principle of LoRa.....	9
3.1.3. Characteristics of LoRa technology	10
3.1.4. Advantages and disadvantages of LoRa technology	10
3.2. LoRa Ra-02 (SX1278) hardware module	12
3.3. Theoretical model for distance estimation via RSSI	14
3.4. ESP32 Development Kit.....	14

3.5. Environmental sensor system	16
3.6. Serial peripheral interface (SPI) protocol	17
3.6.1. Hardware bus architecture	17
3.6.2. Synchronous bit shifting mechanism and operating modes	18
3.7. Inter integrated circuit (I2C) two wire serial protocol	20
3.7.1. Hardware architecture and open drain circuit principle	20
3.7.2. Data frame structure and physical addressing	20
3.8. Firebase realtime database cloud platform	21
3.9. Long short term memory (LSTM) deep learning algorithm	22
3.9.1. Introduction to LSTM	22
3.9.2. The long term dependency problem in standard RNNs	23
3.9.3. Architecture of an LSTM cell	24
3.9.4. Structure and operation of LSTM	24
3.9.5. Detailed architecture of an LSTM node	26
3.9.6. Multivariate time series forecasting strategy in the system	30
CHAPTER 4: SYSTEM FUNCTIONS	32
4.1. Real time environmental parameter monitoring function	32
4.1.1. Multi point distributed monitoring	32
4.1.2. Real time synchronization	32
4.2. Multi mode integrated device control function	32
4.2.1. Manual control mode	32
4.2.2. Threshold based automatic control mode (Auto mode)	33
4.2.3. Predictive intelligent AI control mode (AI mode)	34
4.3. Network topology management and dynamic node configuration function	35
4.3.1. Network scanning and new node addition function	35
4.3.2. Node deletion and deactivation function	35
4.3.3. Relative localization and network quality monitoring function	36
4.4. Time series data management, preprocessing, and logging function	36

4.5. Interaction and intelligent alert notification function on the web UI	37
4.6. Safety and system protection functions	37
CHAPTER 5: DETAILED DESIGN	40
5.1. Hardware design	40
5.1.1. Component list	40
5.1.2. Schematic diagrams	42
5.2. Algorithm and processing flow design	45
5.2.1. Processing logic at the sensor nodes	45
5.2.2. Processing logic at the central coordination station (gateway master)	47
5.2.3. LoRa packet transmission and reception logic	49
5.2.4. RSSI processing and distance estimation algorithm at the master	49
5.3. Database design (firebase structure)	50
5.4. User interface design (web dashboard)	51
5.5. LSTM model	52
5.5.1. Predict phase – soil moisture forecasting	52
5.5.2. Classify phase – soil status classification and decision making	55
CHAPTER 6: SYSTEM IMPLEMENTATION	56
6.1. Hardware and software deployment environments	56
6.2. Physical construction and hardware assembly	57
6.3. Cloud firebase database installation and configuration	58
6.4. AI process and data logging deployment on the edge server	58
6.5. Web dashboard deployment and synchronization	59
CHAPTER 7: EXPERIMENTAL RESULTS AND EVALUATION	61
7.1. Overview of achieved results and practical deployment status	61
7.1.1. Hardware deployment	61
7.1.2. LoRa communication network	62
7.1.3. Cloud synchronization	62
7.1.4. Management interface (web dashboard)	63

7.1.5. Edge AI system	63
7.2. Quantitative empirical evaluation of maximum transmission distance.....	64
7.3. Performance evaluation of the AI model (LSTM)	65
7.3.1. Convergence and error margin evaluation	66
7.3.2. Real time inference testing results	68
CHAPTER 8: LIMITATIONS AND FUTURE DIRECTIONS	69
8.1. Limitations of the current system	69
8.2. Future directions	70
8.2.1. Algorithm upgrades and ai model optimization	70
8.2.2. Hardware optimization and field validation.....	71
8.2.3. Security enhancements and computer vision integration	71

LIST OF ABBREVIATIONS

No	Acronym	Full form
1	IoT	Internet of Things
2	LoRa	Long Range
3	AI	Artificial Intelligence
4	LSTM	Long Short-Term Memory
5	ESP32	SoC by Espressif
6	P2P	Peer-to-Peer
7	NoSQL	Not Only SQL
8	CSV	Comma-Separated Values
9	UI / UX	User Interface / User Experience
10	GSM	Global System for Mobile Communications
11	LTE	Long Term Evolution
12	WSN	Wireless Sensor Network
13	LoRaWAN	Long Range Wide Area Network
14	I/O	Input/Output
15	JSON	JavaScript Object Notation
16	WAN	Wide Area Network
17	LPWAN	Low-Power Wide-Area Network
18	CSS	Chirp Spread Spectrum
19	SNR	Signal-to-Noise Ratio
20	ISM	Industrial, Scientific and Medical
21	AES	Advanced Encryption Standard
22	VDC	Volts Direct Current
23	GND	Ground
24	RST	Reset
25	DIO	Digital Input/Output
26	SCK	Serial Clock
27	MISO	Master Input Slave Output
28	MOSI	Master Output Slave Input
29	NSS / CS	Negative Slave Select / Chip Select
30	SPI	Serial Peripheral Interface
31	RSSI	Received Signal Strength Indicator
32	SRAM	Static Random Access Memory
33	ROM	Read-Only Memory
34	BLE	Bluetooth Low Energy
35	ADC	Analog-to-Digital Converter
36	DAC	Digital-to-Analog Converter

37	UART	Universal Asynchronous Receiver-Transmitter
38	PWM	Pulse Width Modulation
39	I2C	Inter-Integrated Circuit
40	SCL	Serial Clock Line
41	SDA	Serial Data Line
42	ACK / NACK	Acknowledge / Negative Acknowledge
43	RNN	Recurrent Neural Network
44	BPTT	Backpropagation Through Time
45	RAW	Readily Available Water
46	AJAX	Asynchronous JavaScript and XML
47	API	Application Programming Interface
48	MSE	Mean Squared Error
49	RMSE	Root Mean Squared Error
50	IDE	Integrated Development Environment
51	CDN	Content Delivery Network
52	PDR	Packet Delivery Rate
53	ARIMA	AutoRegressive Integrated Moving Average
54	GRU	Gated Recurrent Unit
55	DLI	Daily Light Integral
56	CNN	Convolutional Neural Network

LIST OF TABLES

Table 3.1. Detailed technical specifications of the LoRa Ra-02 module.....	13
Table 3.2. Pin function description of the Ra-02 module.	13
Table 3.3. Detailed technical specifications of the ESP32 CP2102-M Kit.....	15
Table 3.4. Classification of SPI operating modes.	19
Table 4.1. Functional summary by system components	39
Table 5.1. Component list of the system.	40
Table 5.2. Pin mapping table of gateway (master) components.	43
Table 5.3. Pin mapping table of sensor node components part 1.....	44
Table 5.4. Pin mapping table of sensor node components part 2.....	44
Table 5.5. Soil status classification and decision making matrix.....	55

LIST OF FIGURES

Figure 2.1. Overall system diagram.	5
Figure 2.2. Detailed system architecture diagram.....	6
Figure 3.1. Real image of the 433 MHz LoRa SX1278 Ra-02 module.	12
Figure 3.2. Pinout diagram of the 433 MHz LoRa SX1278 Ra-02 module.	12
Figure 3.3. Real image of the ESP32 – CP2102-M development kit.	15
Figure 3.4. Pinout diagram of the ESP32 – CP2102-M development kit.	15
Figure 3.5. DHT11 temperature and humidity sensor module.....	16
Figure 3.6. GY-30 BH1750 light intensity sensor module.	16
Figure 3.7. Capacitive soil moisture sensor v1.2 module.	17
Figure 3.8. SPI bus interface connection.	18
Figure 3.9. Operational diagram of SPI communication.	19
Figure 3.10. SPI operating modes waveforms.	19
Figure 3.11. I2C data frame structure.	21
Figure 3.12. Overall architecture of an LSTM network.....	22
Figure 3.13. Structural model of an LSTM network.....	25
Figure 3.14. Internal architecture of an LSTM network.	25
Figure 3.15. Operational structure of the forget gate.	27
Figure 3.16. Operational structure of the input gate.	29
Figure 3.17. Operational structure of the output gate.	30
Figure 5.1. Gateway (master) schematic diagram.....	42
Figure 5.2. Sensor node schematic diagram.....	43
Figure 5.3. Processing logic at the sensor nodes.....	46
Figure 5.4. Processing logic at the central coordination station (gateway master).....	48
Figure 5.5. Flowchart of the LoRa packet transmission and reception process.....	49
Figure 5.6. Tree architecture storage layout on firebase database.	51
Figure 5.7. Data processing workflow and structural layers of the deep learning LSTM network in the system.....	54

Figure 6.1. Arduino IDE logo.	56
Figure 6.2. Technical components packaged inside the prototype enclosure.....	58
Figure 7.1. Gateway master and a sensor node assembly.	61
Figure 7.2. Complete hardware setup featuring 1 gateway master node and 2 slave nodes.....	61
Figure 7.3. Hierarchical NoSQL JSON tree organization structure for data storage on firebase.	62
Figure 7.4. Web dashboard user interface managing the distributed sensor nodes.	63
Figure 7.5. Training performance and loss curve evaluation of the LSTM model.....	66

ABSTRACT

In the era of Agriculture 4.0, automated environmental monitoring and control play a pivotal role in optimizing crop yields. However, traditional wireless solutions like Wi-Fi and Bluetooth suffer from short transmission ranges and high power consumption. Long Range (LoRa) technology emerges as an optimal alternative for agricultural applications due to its long range communication capability and low power consumption. Combining LoRa with Artificial Intelligence (AI) to predict crop demands represents an essential advancement toward intelligent farming systems.

This thesis designs and implements a multi point environmental monitoring and device control system based on a private LoRa Peer-to-Peer (P2P) network. The system tracks key parameters: air temperature, air humidity, light intensity, and soil moisture and controls actuators such as water pumps and cooling fans. Notably, a Long Short Term Memory (LSTM) deep learning architecture is integrated to forecast soil moisture trends, thereby optimizing water resource allocation. The hardware architecture utilizes the ESP32 microcontroller as the central processing unit interfaced with the Ra-02 (SX1278) LoRa module and a standard sensor suite (DHT11, BH1750, and capacitive soil moisture sensors). The prototype is deployed as a mini greenhouse consisting of one Gateway Master and two Slave Nodes to collect time series datasets in a real world environment. Data synchronization is achieved via Firebase Realtime Database, which feeds a Python based AI engine and a responsive Web Dashboard.

Experimental results demonstrate stable operation, low latency cloud synchronization, and seamless multi mode control (Manual, Auto, and AI). The system establishes a robust IoT infrastructure for precision agriculture. Future extensions will focus on longitudinal data collection to improve LSTM accuracy, scaling the network topology to hundreds of edge nodes, and integrating renewable solar energy for outdoor deployment.

Keywords: smart agriculture, LoRa P2P, ESP32, Firebase, LSTM network

PREFACE

For decades, agriculture has played a vital role as a spearhead economic sector and a major export driver for Vietnam. However, under the impacts of climate change and water scarcity, traditional experience based farming methods are showing significant limitations, leading to resource waste and unstable crop yields.

With the rapid advance of the Fourth Industrial Revolution, smart agriculture has become an inevitable trend. Currently, although various IoT solutions are widely applied, major technical barriers remain: Wi-Fi and Bluetooth connections are limited by short transmission ranges, while cellular networks consume excessive energy. Furthermore, most existing systems only monitor real time parameters and lack the predictive capabilities needed for proactive resource management.

To address these real world challenges and provide a highly practical technological solution, I chose the thesis topic: “Design and implementation of a smart agriculture monitoring and control system using LoRa network integrated with AI based environmental forecasting.”

This project focuses on building a low power, long range wireless sensor network based on LoRa technology to achieve multi point connectivity. A core highlight of this research is the integration of a Long Short Term Memory (LSTM) deep learning model to forecast soil moisture trends, enabling the system to make proactive irrigation decisions.

This thesis serves as a valuable opportunity for me to apply the professional knowledge gained over four years at the university. It is also an essential preparation step for my future career development in the field of Embedded Systems and the Internet of Things (IoT).

CHAPTER 1: THESIS OVERVIEW

1.1. Research necessity

In recent years, smart agriculture has become an essential part of the Fourth Industrial Revolution to cope with the challenges of climate change and global water scarcity [12]. Accurately monitoring and managing environmental conditions, such as temperature, air humidity, and light intensity, is a decisive factor for crop yield and quality in modern farming models [18].

However, traditional agricultural methods based on empirical intuition or primitive on site monitoring systems still show many limitations in terms of timeliness, reliability, and overall management efficiency [13]. The lack of continuous big data models prevents operators from providing adaptive responses to the non-linear variations of microclimates, leading to water resource waste and increased labor costs [16].

1.2. Problem statement

Legacy agricultural IoT systems often rely on short range wireless connections such as Wi-Fi or Bluetooth. These protocols face significant obstacles due to short physical transmission ranges and poor obstacle penetration under the impact of dense tree canopies and iron frame structures in large scale greenhouses or farmlands [11].

On the other hand, cellular broadband network architectures (GSM/3G/4G LTE), while providing large coverage areas, consume very high operating energy, involve complex circuit structures, and incur high periodic subscription costs for each sensor node [1]. Therefore, the problem addressed in this thesis is to research and design an optimal wireless communication solution that balances three core technical criteria: long transmission range, ultra low power consumption at edge nodes, and optimized infrastructure investment costs [14].

1.3. Literature review

To solve the communication and automation problems in agriculture, various approaches globally and in Vietnam have been experimentally deployed. For instance, a study by Srbinovska et al. (2015) implemented a Wireless Sensor Network (WSN) based on the Zigbee standard for microclimate monitoring [20]. The results showed that the system saved energy relatively well; however, the link range between network nodes was limited to a radius of under 100 meters, requiring the installation of many complex repeater stations when expanding the farm area. Another study by Khanna and Kaur (2019) applied the industrial LoRaWAN wide area network protocol connected to centralized Cloud systems for large scale farm management [12]. This architecture solves the distance problem but relies entirely on expensive industrial Gateway infrastructure, high telecommunication licensing fees, and a LoRaWAN packet transmission frame structure that increases I/O latency, making it unoptimized for small and medium scale greenhouse models that require urgent responses. Regarding domestic research, most current agricultural IoT studies only stop at collecting instantaneous environmental parameters and making actuator switching decisions based on fixed thresholds [1]. This mechanism is reactive to incidents, preventing the system from handling the physical delay of water infiltration into the soil, which leads to suboptimal irrigation [21].

Based on this comprehensive overview, this project aims to fill research gap by designing a minimalist LoRa P2P network architecture that operates locally offline without relying on network provider infrastructure, thereby optimizing hardware investment costs. Notably, the distinct advantage of this thesis is the integration of a deep learning data stream using a multivariate LSTM network running on a Cloud edge server. The system completely shifts from the conventional automatic system's state of "waiting for the soil to dry before watering" to a state of "forecasting drought trends 30 minutes in advance to proactively regulate interval irrigation," fully optimizing water resources and the biological productivity of the plants.

1.4. Research objectives

The project focuses on designing and constructing a multi point monitoring and control system for smart agriculture with the following specific objectives:

- Establish a wireless communication network based on LoRa technology to connect sensor stations over long distances.
- Design Slave stations capable of collecting data from a suite of three sensors: air temperature/humidity, light intensity, and soil moisture.
- Develop a Gateway station (Master) to aggregate data and synchronize it with the Cloud Firebase system.
- Build a Web Dashboard that allows users to monitor and control devices in real time.
- Integrate an AI model to enhance the system's automated decision making capabilities.

1.5. Significance of the thesis

- Practical significance: Provides an efficient automated monitoring and irrigation solution, helping save water and labor costs while minimizing risks to crops through AI forecasting capabilities.
- Academic significance: Offers an opportunity to systematize knowledge in embedded systems, wireless networks, and the practical application of artificial intelligence. The research results contribute to the repository of agricultural IoT solutions at VNU-VJU.

CHAPTER 2: SYSTEM ARCHITECTURE

2.1. Overall System Architecture

The smart greenhouse monitoring and control system is designed with a strict layered architecture. This structure ensures data separation and data integrity among the hardware layer, the wireless transmission layer, and the smart processing layer [16]. The entire logical architecture is based on a traditional star network topology, where the Gateway Master station acts as the central coordinator [14].

The system consists of three main components:

- End devices layer (End Nodes - Slaves): Interacts directly with the physical environment to collect data and execute local tasks.
- Coordination and relay layer (Gateway - Master): Manages the local wireless network and handles the Internet connection for data synchronization.
- Application and AI Layer (Cloud & AI Server): Stores system data and makes intelligent control decisions based on predictive models.

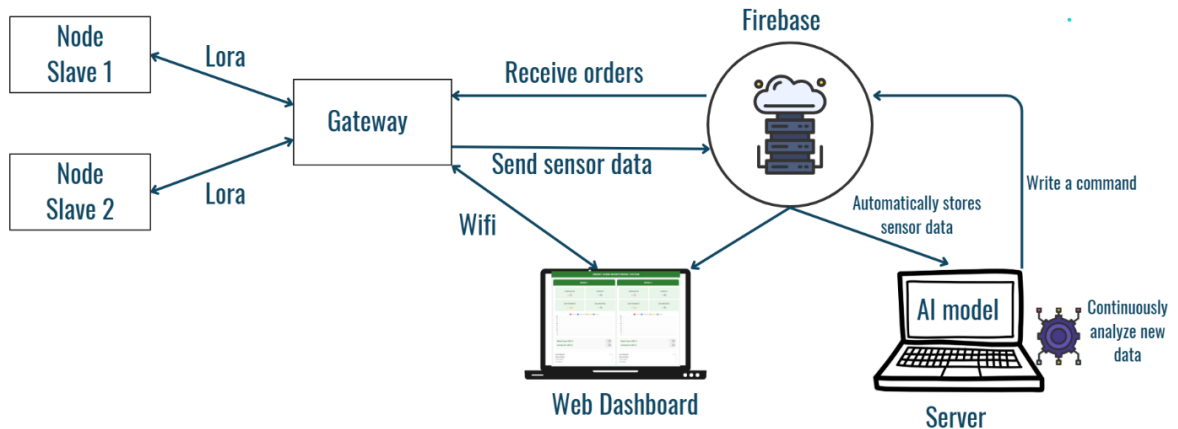


Figure 2.1. Overall system diagram.

2.2. Detailed system architecture diagram

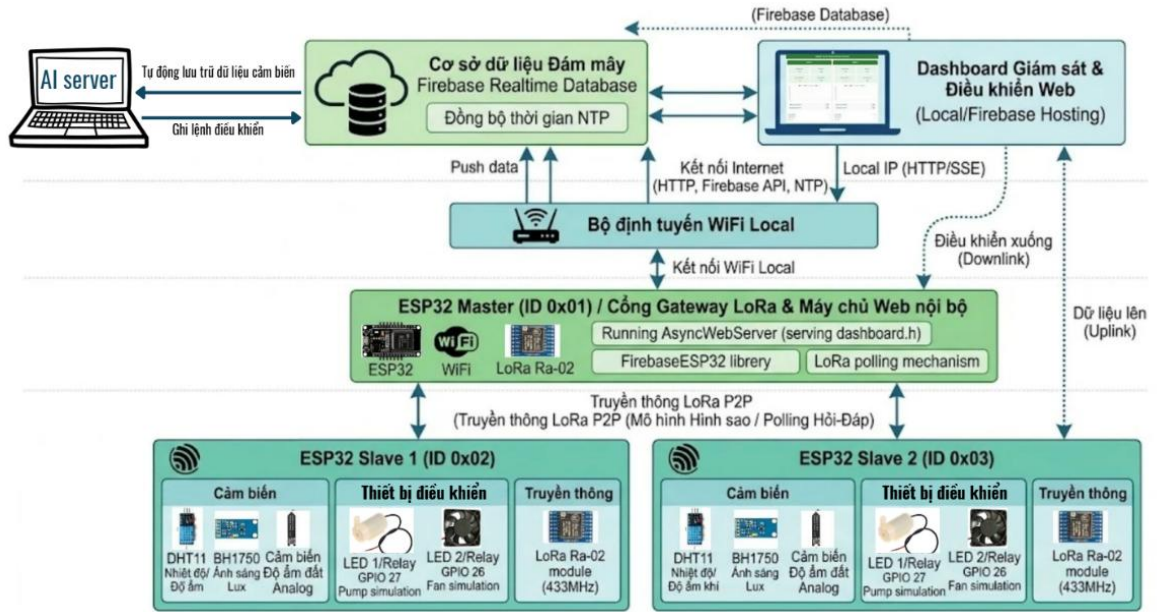


Figure 2.2. Detailed system architecture diagram.

As shown in the detailed system architecture diagram in Figure 2.2, the entire system is organized and operated based on a star network topology, which includes the following specific functional blocks:

2.2.1. Field model block

This edge hardware layer consists of independent distributed sensor stations performing multi point monitoring:

- Slave stations (Nodes 1 & 2): Built around an ESP32 microcontroller for local data processing and coordination.
- Uplink sensors: A suite comprising a DHT11 (ambient temperature/humidity via digital interface), a BH1750 (light intensity via I2C), and a capacitive sensor (soil moisture via analog ADC inputs).
- Downlink actuators: Peripheral devices (a water pump and a cooling fan) connected to the ESP32 through isolated relay switching circuits.

- Wireless communication: An integrated Ra-02 (SX1278) module acting as the transceiver for long range LoRa messaging.

2.2.2. Central coordination gateway block (master node)

The master node coordinates the local wireless loop and bridges data two-way:

- Internal LoRa P2P network: Operates a sequential polling routine over 433 MHz to query specific Slave addresses, completely eliminating radio channel collisions.
- Wide area network (WAN): Utilizes the ESP32's built-in Wi-Fi module to connect to a local internet access point, bridging the field network with the cloud.

2.2.3. Intermediate cloud Infrastructure block (firebase cloud)

Firebase realtime database: A centralized NoSQL database that stores sensor metrics and operational control flags as JSON strings, providing instant data synchronization across all connected clients.

2.2.4. Application and AI layer

Two software entities run in parallel and interact directly with Firebase Cloud:

- Web dashboard: Built on standard HTML5/CSS3 and vanilla JavaScript. It uses Chart.js for real time line charts and offers dropdown menus and toggle switches for remote monitoring and mode switching (MANUAL, AUTO, AI).
- AI server (Edge Server): Runs background Python processes, including a data logger script that compiles clean historical telemetry into .csv files. A TensorFlow/Keras based LSTM model executes multivariate time series inference to predict soil moisture 30 minutes in advance, writing the resulting logical command (ai_decision) back to Firebase.

2.3. Data flow and control mechanism

2.3.1. Monitoring data flow (Uplink)

Environmental sensors → Slave Node processing → LoRa P2P transmission → Gateway Master reception (decodes payload and extracts RSSI for distance estimation) → Firebase Cloud synchronization under targeted paths (/Node1 and /Node2).

2.3.2. Control command flow (Downlink)

The system manages operations through three flexible control scenarios:

- Manual control mode: User toggles UI switch → Firebase state update → Master detects variable change → LoRa command to target Slave → Relay triggers actuator.
- Threshold based automatic mode: The gateway master independently executes localized rule based logic. This localized loop ensures continuous greenhouse safety even during a WAN internet disconnection or an edge server failure.
- Predictive AI mode: The edge AI server processes a historical sliding window to forecast soil moisture 30 minutes into the future. If a risk threshold is crossed, the server writes a control command to firebase, prompting the master to trigger proactive irrigation without human intervention.

2.4. Communication Network Structure

Choosing a star network topology combined with the LoRa P2P protocol yields high energy efficiency and simple scalability [14]. The Slave stations do not communicate with each other; they only exchange data directly with the central master. This design completely eliminates complex routing algorithms, reduces the memory and processing load on the ESP32 embedded chip, and allows the edge nodes to maximize deep sleep cycles to extend battery life [5].

CHAPTER 3: TECHNOLOGICAL FOUNDATIONS

3.1. LoRa communication technology

3.1.1. Introduction to LoRa technology

LoRa, short for long range radio, is a low power wide area network (LPWAN) wireless communication technology operating at the physical layer. It is designed to allow IoT devices to transmit low-data-rate streams over long distances up to tens of kilometers without requiring bulky power amplifiers [14].

LoRa technology simplifies radio transceiver circuits and significantly optimizes power consumption efficiency [5]. Therefore, this protocol is an ideal choice for distributed wireless sensor network (WSN) architectures in large-scale agriculture, enabling edge nodes to operate reliably for several years on a limited battery power supply [13]. This technology was originally developed by Cycleo and was later acquired and commercially standardized by Semtech in 2012.

3.1.2. Operational principle of LoRa

The core technical foundation of LoRa technology is based on the chirp spread spectrum (CSS) modulation method [5]. Under this method, original digital data is not transmitted using fixed frequencies to avoid boundary interference. Instead, it is broken down into high-frequency pulses and encoded into a sequence of chirp signals (sinusoidal signals whose frequency continuously varies over time) before radiating through the antenna [14].

There are two types of chirp signals:

- Up-chirp: The frequency increases over time.
- Down-chirp: The frequency decreases over time.

The operational principle of spreading energy across a wide bandwidth helps the receiver circuit decode data with extremely high accuracy [5]. The system can automatically filter

noise and fully demodulate the packet even when the radio signal strength is severely attenuated, dropping well below the noise floor of the surrounding environment (signal to noise ratio $SNR < 0$) [14].

The operational frequency bands of LoRa technology range from 430MHz to 915MHz, applied to different regions worldwide. Specifically, the 433MHz band is allocated for Asia, the 868MHz band for Europe, and the 915MHz band for North America [5]. The chirp signal allows multiple LoRa devices to operate and exchange data simultaneously over multiple channels in the same area without causing overlap or mutual signal interference.

3.1.3. Characteristics of LoRa technology

The LoRa wireless communication network possesses outstanding technical characteristics, fully meeting the requirements of specialized IoT systems:

- Exceptional communication range: It achieves a link distance of 2 km to 5 km in high-density urban environments with high obstacle density, and can reach up to 10 km to 15 km in open spaces or open agricultural fields [13].
- Optimized power efficiency: Edge nodes consume an ultra-low static current (only a few μA in deep sleep mode), which extends the operating battery life for up to several years [5].
- Low data rate for extended range: The data transmission rate of LoRa is limited between 0.3 kbps and 50 kbps [14]. Although this speed is unsuitable for multimedia transmission (such as video or audio), it provides sufficient bandwidth for packaging small-sized agricultural sensor data packets [13].

3.1.4. Advantages and disadvantages of LoRa technology

Although LoRa is a highly popular technology, it is not perfect in every aspect.

Advantages

The prominent advantages of LoRa technology compared to other connectivity methods in the same field include:

- Cost effective long range: Strong obstacle penetration and high immunity to multipath interference significantly reduce deployment costs by eliminating the need for expensive repeaters [14].
- Low power and operating costs: Low power consumption extends battery life, minimizing the frequency of hardware maintenance and battery replacements for farmers [5].
- Security capability: The LoRaWAN standard natively supports AES128 encryption to enable mutual authentication and ensure data integrity. However, within the scope of this thesis, only unencrypted LoRa P2P communication is implemented, leaving encryption as a future development objective.
- High capacity and standardization: As an open network protocol, it supports rapid multi-device deployment, enabling a single gateway to collect millions of messages for robust data tracking.

Disadvantages

Although LoRa technology has many excellent applications in the IoT trend, it still has certain disadvantages to consider:

- Low bandwidth and small payloads: Strictly limited payload sizes (around 100 bytes) and slow data rates result in higher latency, making it unsuitable for continuous, massive data streams.
- Environmental and structural sensitivity: Signal propagation is heavily affected by terrain, signal fading, and physical barriers, creating deployment challenges in obstructed or high-density urban areas.

- Upfront infrastructure costs: Implementing a functional network requires dedicated gateways alongside node connections, which increases initial capital expenses.

3.2. LoRa Ra-02 (SX1278) hardware module

Within the scope of this project, both the central gateway master station and the sensor slave nodes integrate the Ra-02 hardware module manufactured by Ai-Thinker, which is based on the specialized SX1278 radio transceiver chip from Semtech [5].

Characteristics: The module operates in the unlicensed 433 MHz frequency band (the ISM band allocated for Asia). This lower frequency band provides a longer wavelength, resulting in excellent obstacle penetration and outstanding resistance to terrain diffraction. It is fully compatible with environments featuring dense tree canopies and extensive iron frame structures, such as in smart agricultural greenhouses.

Although it features a low data bit rate, the transmission distance can reach several kilometers, completely meeting the requirements of multi-point monitoring in wide-area farms.

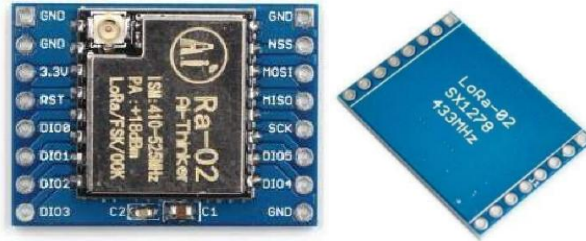


Figure 3.1. Real image of the 433 MHz LoRa SX1278 Ra-02 module.

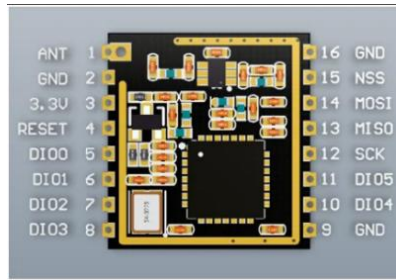


Figure 3.2. Pinout diagram of the 433 MHz LoRa SX1278 Ra-02 module.

Table 3.1. Detailed technical specifications of the LoRa Ra-02 module.

Technical parameter	Physical configuration value
Standard operating voltage	2.5V-3.7V
Current consumption during transmission (Tx)	120mA
Current consumption during reception (Rx)	10.8mA
Standby current consumption	0.2uA
Output power	20dbm (100mW)
Configuration communication interface	SPI
Supported modulation modes	FSK, GFSK, MSK, GMSK, LoRaTM, OOK
Sensitivity	+20dBm – 10dBm
Frequency band	420 – 450Mhz
Theoretical communication distance	Up to 8,000 m (In open space conditions)
Transmission rate	300 kbps (default 2.4 kbps)
Number of pins	16
Dimensions	21mm*36mm
Operating temperature	-40°C – 85°C

Table 3.2. Pin function description of the Ra-02 module.

Pin name	Signal type	Function
GND	Power	0 VDC ground reference
3.3V	Power	Provides a stable 3.3V DC power supply
RST	Input	System hardware reset
DI00 – DI05	Input/Output	General digital I/O pins for signal transmission/reception and hardware interrupts
SCK	Input	Serial clock for SPI communication
MISO	Output	Master input slave output (data output channel for the SPI bus)
MOSI	Input	Master output slave input (data input channel for the SPI bus)
NSS	Input	Chip select / slave select (active LOW to enable SPI communication)

3.3. Theoretical model for distance estimation via RSSI

In LoRa wireless communication, the RSSI (Received signal strength indicator) represents the received signal strength at the antenna of the receiving station (gateway), measured in dBm. The RSSI value is inversely proportional to the transmission distance due to path loss in space.

To convert the RSSI value from dBm into the physical distance d (meters), this project applies the "Log distance path loss model".

The general mathematical formula is defined as follows:

$$\text{RSSI} = \text{RSSI}_0 - 10n \log_{10} \left(\frac{d}{d_0} \right) + X_\sigma$$

Where:

- d : The estimated distance between the transmitter (slave) and the receiver (master).
- d_0 : The standard reference distance.
- RSSI_0 : The actual signal strength measured at the reference distance d_0 .
- n : The path loss exponent, which depends on the specific environmental obstacles.
- X_σ : A Gaussian random noise component caused by multipath fading phenomena.

By neglecting the random noise component to optimize calculations on the microcontroller, the empirical mathematical formula used to extract the distance d is transformed into:

$$d = d_0 * 10^{\left(\frac{\text{RSSI}_0 - \text{RSSI}}{10n} \right)}$$

3.4. ESP32 Development Kit

The processing core of all hardware stations in the system is the ESP32 microcontroller, a powerful System-on-Chip (SoC) featuring integrated dual mode wireless connectivity developed by Espressif Systems [1]. Specifically, this project uses the commercial ESP32 Wi-Fi + Bluetooth CP2102-M development kit.



Figure 3.3. Real image of the ESP32 – CP2102-M development kit.

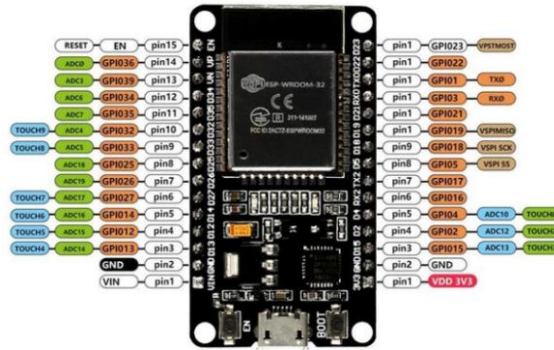


Figure 3.4. Pinout diagram of the ESP32 – CP2102-M development kit.

Table 3.3. Detailed technical specifications of the ESP32 CP2102-M Kit.

Technical parameter	Physical configuration value
Microprocessor	Xtensa dual-core 32-bit LX6
Operating Frequency	240MHz
USB-to-Serial Chip	CP2102
SRAM	512KB
ROM	448 Kbyte
Supply Voltage (USB)	5V DC
Voltage input/output	3.3V DC
Power Consumption	5 μ A in deep sleep mode
Performance	Up to 600 DMIPS
Wireless Connectivity	802.11 b/g/n Wi-Fi standard
Bluetooth	v4.2 BR/EDR and BLE
Peripherals	18-channel 12-bit ADC and 2-channel 8-bit DAC
Synchronous Interfaces	SPI, I2C, I2S
Asynchronous Interfaces	UART, PWM
Hardware Crypto Acceleration	AES, SHA-2, RSA, ECC, RNG

Applications:

- Dual wireless connectivity: By combining both Wi-Fi and bluetooth on a single development board, the ESP32 can simultaneously transmit and receive control signals to and from other devices using both wireless protocols.
- Optimized power consumption: The ESP32 is specifically engineered for mobile hardware, remote sensors, and diverse IoT applications. It features robust operational stability alongside high electromagnetic noise immunity.

3.5. Environmental sensor system

The system integrates three standard sensors to collect key environmental and biological parameters:

DHT11 sensor: Measures ambient temperature and relative humidity, which directly influence plant transpiration and photosynthesis [14].



Figure 3.5. DHT11 temperature and humidity sensor module.

BH1750 (GY-30) sensor: Measures light intensity (Lux) via an internal 16-bit ADC and I2C interface, allowing the AI engine to evaluate solar radiation and forecast soil water depletion [14].



Figure 3.6. GY-30 BH1750 light intensity sensor module.

Capacitive soil moisture sensor: Measures volumetric soil water percentage. Utilizing a corrosion-resistant coating for high durability over resistive alternatives, it provides the core quantitative baseline for automated irrigation decisions [21].

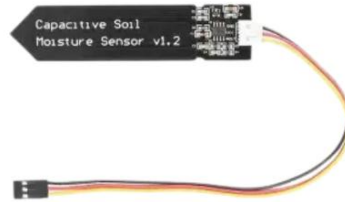


Figure 3.7. Capacitive soil moisture sensor v1.2 module.

3.6. Serial peripheral interface (SPI) protocol

SPI communication is a synchronous serial data transmission protocol operating in full-duplex mode, designed by Motorola to connect high-speed peripheral chips [2]. In this system, the SPI bus acts as the primary data pipeline for transmitting accurate configuration data between the central ESP32 microcontroller and the SX1278 radio frequency chip on the Ra-02 module.

3.6.1. Hardware bus architecture

The SPI standard requires a strict 4-wire physical interface between the master device (master - ESP32) and the slave device (slave - Ra-02) [2]:

- SCK (serial clock): The clock signal line actively generated by the ESP32 to synchronize the timing of data bit shifting. The data transmission rate is directly determined by the frequency of this SCK signal.
- MOSI (master output slave input): The unidirectional data line used by the ESP32 master to send configuration commands or raw packet payloads down to the LoRa module.

- MISO (master input slave output): The unidirectional data line used by the LoRa module to send data from its internal FIFO buffer back to the master microcontroller for processing upon receiving an incoming wireless packet.
- NSS/CS (chip select): The control line used to manage device selection. In its idle state, the NSS pin is held HIGH (3.3V). When the ESP32 initiates a read or write data cycle with the LoRa module, it pulls the NSS line LOW (0V) to activate the internal logic engine of the SX1278.

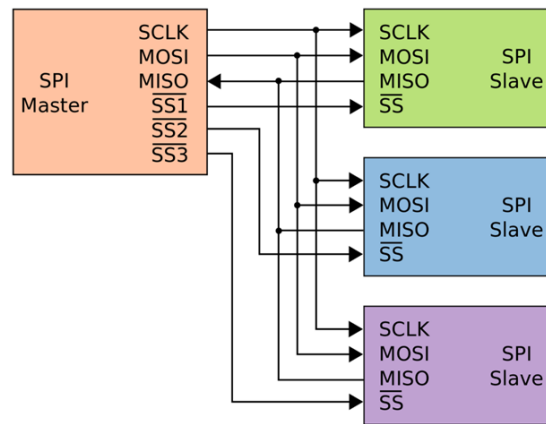


Figure 3.8. SPI bus interface connection.

3.6.2. Synchronous bit shifting mechanism and operating modes

The data exchange process relies on an internal closed-loop architecture featuring two interconnected shift registers [2]. At each clock pulse of the SCK line, 1 bit of data from the Master's shift register is pushed out onto the MOSI line to load into the Slave. Simultaneously, 1 bit from the Slave is pushed out onto the MISO line to load into the master. After exactly 8 clock cycles, a complete byte of data is exchanged in parallel between the two devices.

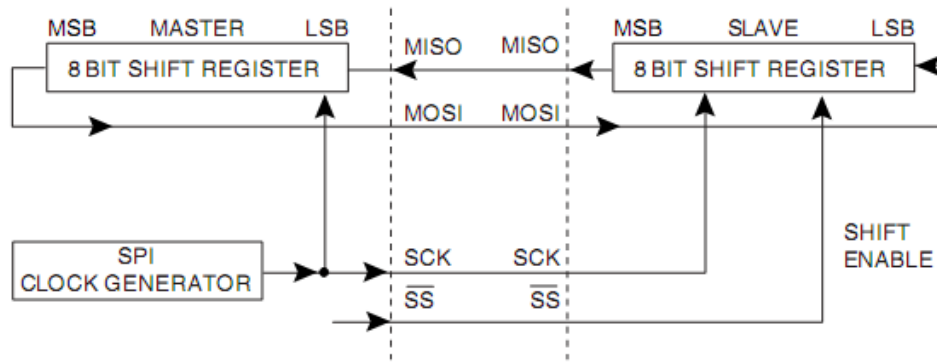


Figure 3.9. Operational diagram of SPI communication.

To ensure hardware compatibility, the SPI protocol defines four operating modes by configuring two parameters: Clock polarity (CPOL) and Clock phase (CPHA) [2].

Table 3.4. Classification of SPI operating modes.

Operating Mode	CPOL Value	CPHA Value	SCK Idle State	Data Sampling Edge
Mode 0	0	0	LOW (0V)	First rising edge of SCK
Mode 1	0	1	LOW (0V)	First falling edge of SCK
Mode 2	1	0	HIGH (3.3V)	First falling edge of SCK
Mode 3	1	1	HIGH (3.3V)	First rising edge of SCK

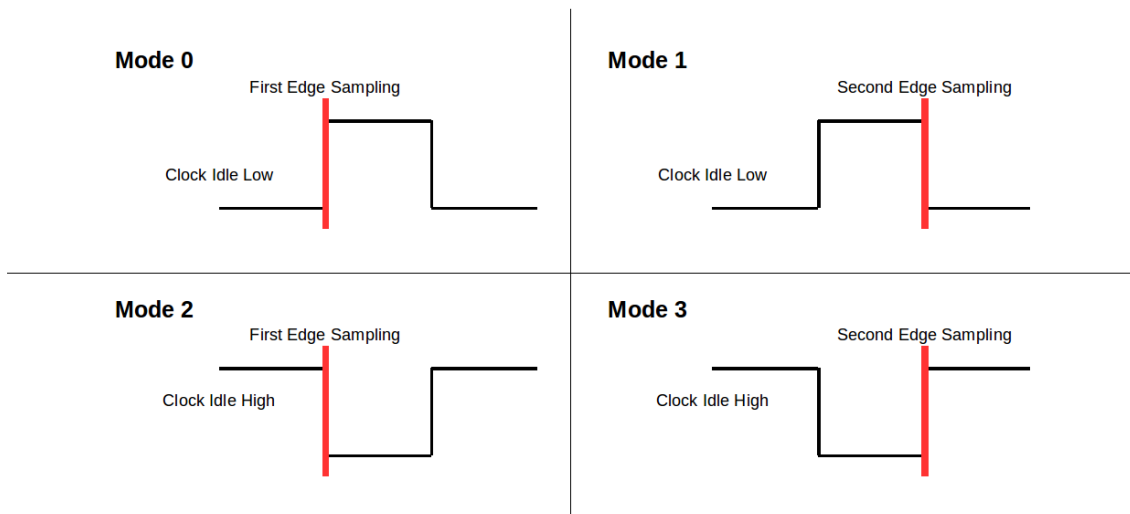


Figure 3.10. SPI operating modes waveforms.

Practical application: The system's LoRa control library is configured by default to operate in SPI Mode 0. This ensures absolute data synchronization accuracy and eliminates bit error rates at high operating frequencies.

3.7. Inter integrated circuit (I2C) two wire serial protocol

For high-quality environmental data collection sensors such as the BH1750 digital light sensor, the system implements the I2C bus communication standard developed by Philips Semiconductors [3]. This is a synchronous serial communication protocol supporting a multi-master multi-slave topology, which minimizes the utilization of microcontroller GPIO pins.

3.7.1. Hardware architecture and open drain circuit principle

I2C communication relies on only two shared signal lines for the entire bus network [3]:

- SCL (Serial clock line): The clock line transmitted by the master device to maintain synchronous transmission timing.
- SDA (Serial data line): The bidirectional data line used to exchange information between the master and the sensors.

Because the hardware structure of both the SCL and SDA lines is designed using an open drain (or open collector) architecture, devices connected to the bus network can only pull the lines LOW (0V) but cannot actively drive them HIGH. Consequently, the system architecture must integrate two pull up resistors (typically valued between 1 k Ω and 4.7 k Ω) to maintain a default HIGH logic state (3.3V) on the bus, preventing short circuits when multiple devices contend for bus access.

3.7.2. Data frame structure and physical addressing

Each slave device connected to the I2C bus network possesses a unique 7-bit physical address pre-assigned by the manufacturer (for example, the BH1750 chip has a

default address of 0x23 or 0x5C). Therefore, the system can manage up to a hundred sensors using only a single pair of pins without any data routing confusion.

A standard I2C data frame follows a specific logical sequence:

1. Start condition: The master pulls the SDA line LOW before the SCL line transitions LOW, signaling the beginning of a bus acquisition cycle.
2. Address block: The master transmits a 7-bit address sequence of the targeted sensor (e.g., 0x23 for the BH1750), followed by an eighth bit determining the read/write mode (R/W bit: 0 for writing configuration commands, 1 for reading data).
3. ACK/NACK acknowledgment bit: The sensor with the matching address responds by pulling the SDA line LOW during the ninth clock cycle to signal its readiness to communicate.
4. Data block: Sensor measurement values or configuration commands are transmitted sequentially in 8-bit blocks. After each byte, an ACK response is returned to verify data integrity.
5. Stop condition: The master transitions the SDA line from LOW to HIGH after the SCL line has transitioned HIGH, fully releasing the bus network for subsequent cycles.

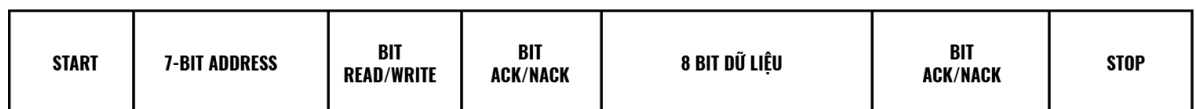


Figure 3.11. I2C data frame structure.

3.8. Firebase realtime database cloud platform

Firebase is a google powered NoSQL cloud database that stores data as JSON structures and synchronizes it across connected clients with minimal latency.

- Real time synchronization: Enables instant data synchronization from the ESP32 gateway master to the web dashboard and AI server without polling mechanisms. It natively supports local caching for automatic offline data recovery.
- Scalable JSON tree topology: The hierarchical structure simplifies data management and scalability for multiple distributed field stations (Node1, Node2, etc.).

3.9. Long short term memory (LSTM) deep learning algorithm

3.9.1. Introduction to LSTM

Long short term memory (LSTM) is an advanced variant of the recurrent neural network (RNN), specifically designed to capture long-term dependencies in time-series data [6]. This model utilizes a dedicated memory cell to store information over time, thereby overcoming the short-term memory limitations of traditional RNNs when retaining and processing long data sequences [10].

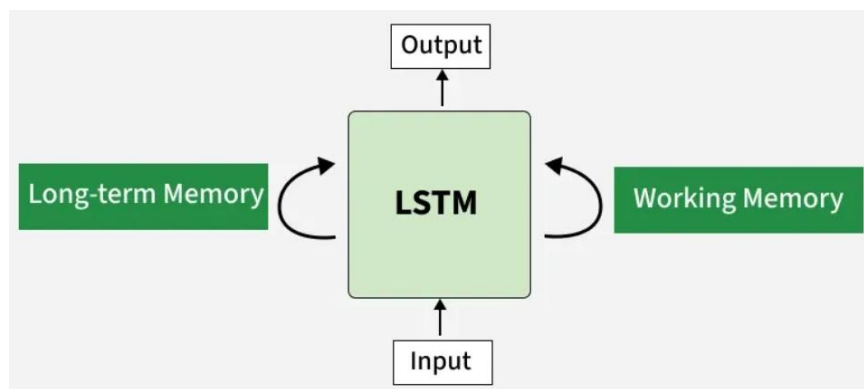


Figure 3.12. Overall architecture of an LSTM network.

This structural design makes LSTM highly effective in the following scenarios:

- Long term dependency processing: It has the capability to retain and link historical information across extensive data sequences.
- Memory cell integration: It securely stores, alters, and updates critical information over long time vectors.

- Superiority over RNN: It successfully resolves the vanishing gradient constraint inherent in legacy RNN systems.
- Diverse applications: Widely applied in fields such as machine translation, speech recognition, and complex time series forecasting.

3.9.2. The long term dependency problem in standard RNNs

Recurrent neural networks (RNNs) are designed to process sequential data by utilizing a hidden state to carry information from previous timesteps. However, standard RNNs struggle to learn long-term relationships due to two primary mathematical challenges:

The main reason are:

- Vanishing gradient problem: During model training across multiple timesteps, the gradients used for updating weights shrink exponentially as they backpropagate through time. This makes it difficult for the model to capture long-term patterns because information from early steps loses its functional impact [4].
- Exploding gradient problem: Conversely, gradients can grow excessively large, causing critical instability during the training process. When this happens, weight updates become erratic and unpredictable, preventing the model from converging stably [4].

To address these gradient calculation bottlenecks of traditional RNNs in sequence processing, the LSTM architecture was introduced by Sepp Hochreiter and Jürgen Schmidhuber [10].

In an agricultural monitoring context, changes in soil moisture occur continuously and are cumulatively influenced by historical microclimate factors (temperature, air humidity, light intensity) over an extended past window. Traditional RNNs lose the ability to link long range temporal data because the chained multiplications of derivatives in the backpropagation through time (BPTT) algorithm drive the weights of early timesteps down to zero.

The LSTM network overcomes this deficiency by introducing a cell state, which acts as a linear conveyor belt running straight through the temporal chain. This conveyor belt uses specialized logic gates to add, remove, or regulate information within the cell state linearly, allowing the model to preserve past environmental trends and output accurate future predictions [6].

3.9.3. Architecture of an LSTM cell

The core architecture of an LSTM unit includes a memory cell regulated by three primary operational gates [10]:

- Input Gate: Controls how much new incoming information is written into the memory cell.
- Forget Gate: Determines which old historical information should be discarded from the memory cell.
- Output Gate: Controls what information from the memory cell is exported to the next layer as a hidden state output.

Through these three internal gates, the LSTM network selectively filters data passing through the node, enabling it to learn long-term dependencies within the time series. Additionally, an LSTM maintains a Hidden State analogous to short term memory which is continuously updated based on the current input, the previous hidden state, and the current cell state..

3.9.4. Structure and operation of LSTM

An LSTM network consists of a repeating chain-like structure composed of memory cells and distinct gating mechanisms.

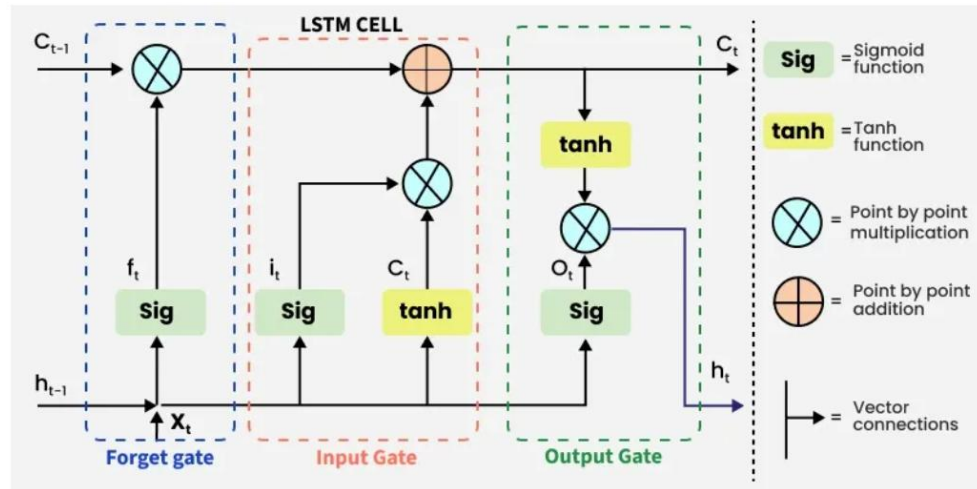


Figure 3.13. Structural model of an LSTM network.

Information is safely stored within these memory cells, while the direct modification and updating of data are executed interactively via the forget gate, input gate, and output gate.

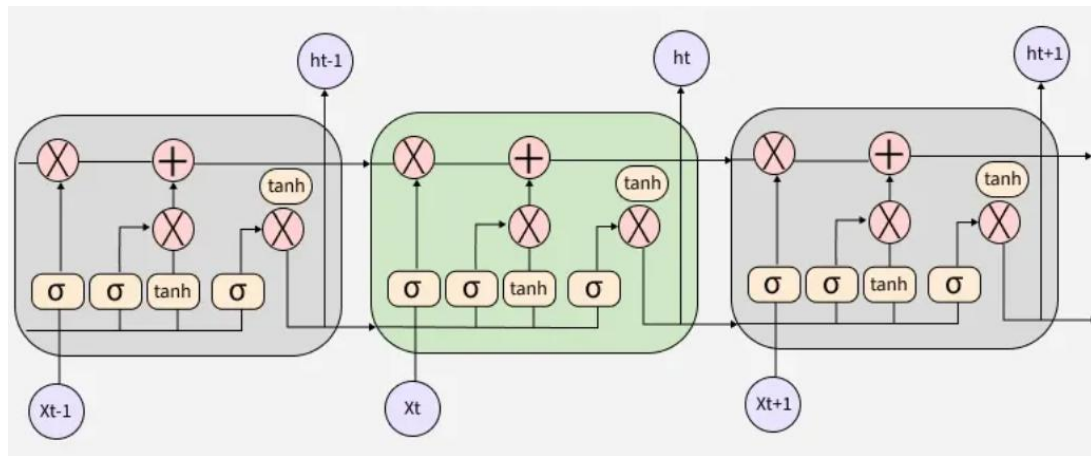


Figure 3.14. Internal architecture of an LSTM network.

Figure 3.14 illustrates how the internal gates and activation functions coordinate to process and store data over time. The three blocks in the diagram (gray – blue – gray) represent three consecutive timesteps $t - 1, t, t + 1$, showing how sequential data flows through the sigmoid (σ) and tanh gates while performing element-wise multiplications and additions to update the hidden state (h) and the cell state (C).

Each unit is an LSTM block with an internal structure containing:

- Sigmoid (σ) functions: Used to control the amount of information to retain or discard.
- Tanh functions: Used to create new candidate values for the memory cell, scaling outputs within the range of $[-1, +1]$.
- Multiplication ($\times$) and addition ($+$) operations: Represent how the gates interact to update the cell state and the hidden state.

The arrows trace the flow of information between timesteps:

- x_t : The input vector at the current timestep.
- h_{t-1}, h_t, h_{t+1} : Hidden states passed across consecutive timesteps.
- Inside each cell, information is processed through three distinct gates:
 1. Forget gate: Decides which information from the previous state should be discarded.
 2. Input gate: Determines what new incoming information should be added to the memory cell.
 3. Output gate: Generates the current output based on the updated cell state.

3.9.5. Detailed architecture of an LSTM node

The core architecture of an LSTM cell consists of an intrinsic cell state C_t and three functional gates activated by non linear functions: the forget gate, the input gate, and the output gate. At each timestep t , the LSTM cell receives the current input vector x_t and the previous hidden state h_{t-1} .

Forget gate

The forget gate decides which information is retained or removed from the cell state. It takes the current input x_t and the previous hidden state h_{t-1} , applies weights and biases, and processes the result through a Sigmoid (σ) activation function to compress the output within the range $[0, 1]$, where:

- A value close to 0 $\rightarrow$ discard the information.
- A value close 1 $\rightarrow$ keep the information.

The mathematical equation for the forget gate is:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Where:

- W_f represents the weight matrix assigned to the forget gate.
- $[h_{t-1}, x_t]$ denotes the concatenation of the current input and the previous hidden state.
- b_f is the bias vector of the forget gate.
- σ is the sigmoid activation function.

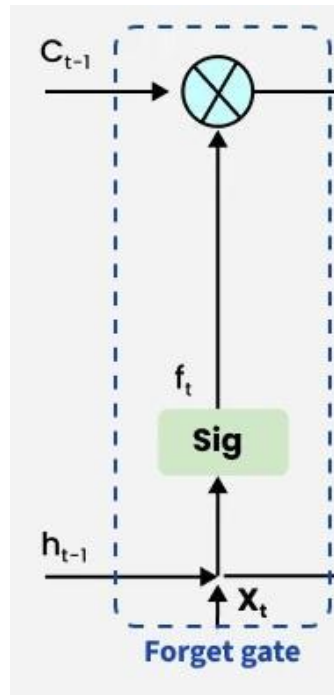


Figure 3.15. Operational structure of the forget gate.

Input gate

The addition of useful information to the cell state is executed by the input gate via a two-step process:

- First, the information is regulated by a sigmoid function to filter which values must be remembered, similar to *forget gate*, using the current input x_t and the previous hidden state h_{t-1} .
- Second, a candidate vector $\tilde{C}_t$ is generated using a tanh function, producing outputs between -1 and +1, to represent all potential new values extracted from the inputs h_{t-1} and x_t .
- Finally, the filtered gate values and the candidate vector are multiplied together to capture the valid information update.

The mathematical equations for the Input Gate and candidate state are:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

The previous cell state C_{t-1} is then multiplied by f_t to drop the discarded historical records, and then added to the adjusted candidate input $i_t \odot \tilde{C}_t$, to formulate the new cell state.

The cell state update equation is defined as:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Where:

- $\odot$ denotes the element-wise multiplication operator.
- Tanh is the activation function.

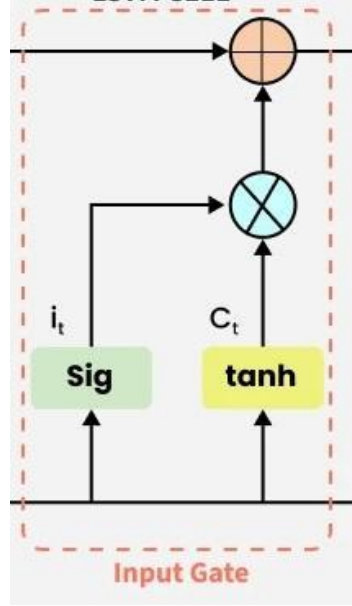


Figure 3.16. Operational structure of the input gate.

Output gate

The output gate decides the next hidden state h_t based on the newly updated cell state. This state functions as both the output of the current layer and the hidden vector passed to the next timestep $t + 1$.

The mathematical equation for the output gate is:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

The new hidden state h_t is subsequently calculated by multiplying o_t with $\tanh(C_t)$:

$$h_t = o_t \odot \tanh(C_t)$$

Where:

- o_t is the activation value of the output gate.
- C_t is the current updated cell state.
- W_f, W_i, W_c, W_o are the weight matrices optimized during model training.
- b_f, b_i, b_c, b_o are the corresponding bias vectors.

Trạng thái ẩn h_t sau đó sẽ được truyền sang bước thời gian tiếp theo ($t + 1$), đồng thời cũng có thể được sử dụng để tạo ra đầu ra của mạng.

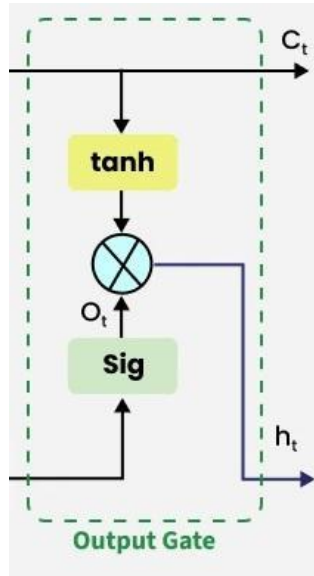


Figure 3.17. Operational structure of the output gate.

3.9.6. Multivariate time series forecasting strategy in the system

Within the scope of this project, the LSTM algorithm is deployed based on a multivariate deep learning network configuration [7]. This setup enables the system to perform complex time-series regression using not only historical soil moisture records but also non linear environmental features acting on the microclimate [17].

Input data block:

The model handles inputs via a fixed size sliding window mechanism structured as a 3D tensor matrix with the following format:

$$\text{Shape} = (\text{Batch Size}, \text{Time Steps}, \text{Features})$$

For each prediction cycle of a targeted node, the system extracts Time Steps = 12 consecutive historical data marks. With a data sampling frequency of 5 minutes per log,

this window represents exactly 60 minutes of historical data. At each step, Features = 4 environmental features are collected:

$$x_t = [\text{Temperature}_t, \text{Humidity}_t, \text{Light}_t, \text{Soil_Moisture}_t]^T$$

Forecasting and decision mechanism:

The core LSTM model processes the (12, 4) input matrix to extract temporal patterns across the past hour. This allows the model to capture the exact water depletion rate of the soil under the direct influence of temperature variations and solar radiation changes [7]. The output layer (a dense layer with a linear activation function) performs linear regression to output a single continuous value:

$$\hat{y}_{t+k} = \text{Soil_Moisture}_{t+30}$$

This 30 minute ahead forecast value is then transferred to a rule based logical classifier to convert the continuous real number into discrete operational control commands (SAFE, WARNING, or PUMP_ON).

The combination of multivariate LSTM time-series forecasting and the automated decision layer establishes an intelligent closed-loop control system. This transition shifts system operations from reactive error handling to proactive resource optimization in smart precision farming [17].

CHAPTER 4: SYSTEM FUNCTIONS

4.1. Real time environmental parameter monitoring function

This is the foundational function that allows the system to digitize the physical state of the mini greenhouse environment and provide structured sensor records from the nodes.

4.1.1. Multi point distributed monitoring

The system simultaneously collects data from independent stations (Node 1 and Node 2) placed at different physical locations in the greenhouse. This function helps operators detect non linear variations in temperature and humidity across different cultivation zones.

Quantitative measurement parameters include: air temperature (°C), relative air humidity (%), ambient light intensity (Lux), soil moisture percentage (%).

4.1.2. Real time synchronization

Once collected over the LoRa network, the data is pushed directly to the firebase realtime database by the gateway master station. The web dashboard utilizes a continuous event listening mechanism to update measurements and line charts with minimal latency.

4.2. Multi mode integrated device control function

The system handles device control (water pump and cooling fan) by integrating a dynamic mode-selection menu directly on the user interface, which is divided into three operational scenarios:

4.2.1. Manual control mode

- Allows operators to fully control the hardware system remotely through the web dashboard user interface, independently of automated algorithms.

- When the configuration menu of the corresponding Node is switched to MANUAL mode, the user can independently turn on or off each relay connected to the water pump or cooling fan using intuitive toggle switches. Control commands from the browser are encoded into minimalist text strings (for example, 1,t to turn on the fan, 2,f to turn off the pump) and transmitted over the wireless LoRa network to optimize the bandwidth usage of the radio channel.

4.2.2. Threshold based automatic control mode (Auto mode)

The gateway master executes localized rule-based logic thresholds, ensuring uninterrupted greenhouse operations during WAN disconnections or edge AI server failures.

Intermittent irrigation algorithm: Triggered when soil moisture drops below 55%. To overcome physical water infiltration delays and prevent sensor errors, the pump runs for exactly 10 seconds, then shuts down for a 3 minute waiting interval. The cycle repeats until soil moisture reaches ($\geq 70\%$).

- 55% and 70% threshold rationale: Derived from precision irrigation literature [18]. The 55% threshold represents the lower limit of readily available water (RAW) to prevent drought stress, while 70% approaches field capacity to avoid root waterlogging.
- 10 second pumping duration: Calibrated via hardware experiments. With the mini DC pump's flow rate (1.5 - 2.0 L/min), a 10-second cycle delivers a precise 250 - 330 ml of water, preventing drainage tray overflow.
- 3 minute waiting interval: Accounts for soil capillary action. Surface water requires 150 - 180 seconds to diffuse uniformly down to the capacitive sensor electrodes. Checking earlier would yield false dry readings, triggering uncontrollable over pumping.

Deadband microclimate control algorithm: The cooling fan activates when ambient temperature exceeds 32°C and deactivates only when it drops below 29°C, preventing rapid on/off cycling at the boundary.

- 32°C threshold rationale: Derived from plant physiology literature [18]. Above 32°C, crops trigger defensive stomatal closure to prevent water loss, which completely halts photosynthesis and induces biological heat shock.
- 3°C deadband rationale (deactivation at 29°C): Optimized through lab trials to match the 5V fan's convection lag. This differential eliminates rapid relay toggling caused by transient thermal fluctuations, protecting the mechanical lifespan of the relay contacts and fan motor while conserving power.

4.2.3. Predictive intelligent AI control mode (AI mode)

This mode shifts the system paradigm from reactive incident response to proactive risk prevention.

- Prediction pipeline: The edge AI server processes a 60 minute historical sliding window of 4 environmental variables to forecast soil moisture 30 minutes into the future using the trained LSTM model.
- Decision execution: The continuous regression output is processed by a logical classifier to determine the optimal control state, which is uploaded to firebase.

The AI warning threshold window is set between 30% and 40%.

Threshold calibration rationale: The AI thresholds (30% - 40% predicted) are intentionally lower than the AUTO thresholds (55% - 70% real time) due to the 30 minute predictive buffer. AUTO mode reacts to immediate dryness, requiring a high threshold (55%) to offset physical water infiltration delays. Conversely, AI mode anticipates trends early; setting a lower activation point (activating the pump below 30%) conserves water, avoids premature irrigation while the soil is safe, and delivers water precisely as the crop approaches the drought boundary.

4.3. Network topology management and dynamic node configuration function

To ensure the system is scalable in practical farming conditions allowing users to customize the number of slave stations based on the greenhouse area the system integrates a remote dynamic network management module..

4.3.1. Network scanning and new node addition function

Operational mechanism: When an operator installs a new sensor station in the field, instead of manually reflashing the firmware on the central gateway master, the user only needs to activate the network scanning function directly on the web dashboard.

Data flow:

1. The browser sends an HTTP GET request to the `/scan_network` API of the gateway master.
2. The master broadcasts a discovery packet over the LoRa network to search for new devices waiting to join the topology.
3. Upon receiving an identification response from the new slave, the master automatically registers that Node's ID into its authorized polling list (whitelist) and simultaneously initializes a new data path for this node on firebase cloud with an initial status of `is_active = true`.

4.3.2. Node deletion and deactivation function

Operational mechanism: In the event that a sensor station experiences a hardware failure, requires maintenance, or is no longer needed in a specific zone, the user can remove the node directly via the web UI without interrupting the operation of other active nodes.

Data flow:

- The user clicks the "Delete" button on the corresponding node card on the web dashboard interface, which triggers a JavaScript function to send a request to the `/delete_node?node=Node_ID` API.

- The gateway master captures the command and immediately updates the status flag to `is_active = false` in the firebase realtime database.
- Polling cycle optimization: During the next data collection cycle in the main `loop()` function, the master checks this `is_active` variable first. Because the status has changed to false, the master immediately skips this node and stops broadcasting the LoRa data request command to it. This mechanism eliminates useless response timeout waiting windows, optimizes wireless network bandwidth, and concentrates hardware resources on other active nodes.

4.3.3. Relative localization and network quality monitoring function

The system not only collects biological data but also continuously monitors the physical condition of the wireless link. Every time the gateway master receives a sensor data packet from a slave node, it automatically measures the packet's RSSI (received signal strength indicator).

Based on the theoretical mathematical model, the system calculates the relative estimated distance (in meters) from that slave station to the central gateway. This metric helps greenhouse operators detect signal blind zones early, evaluate spatial link stability, and quickly locate sensor nodes in the field for maintenance work.

4.4. Time series data management, preprocessing, and logging function

This function handles structured data storage, establishing a solid foundation for machine learning model training.

- Hierarchical cloud storage: The entire historical log of sensor changes is organized chronologically with unique identification keys based on explicit incremental index structures (`latestID`) on firebase.
- Data deduplication: An independent python script running on the server continuously monitors the `latestID` variable of each node. The program only logs and extracts data into a .csv file when it detects an increment in the hardware ID

index. This mechanism completely eliminates the continuous logging of duplicate static data during sensor instability or disconnections, ensuring the time series dataset always achieves the highest level of purity and standardization for neural network training.

4.5. Interaction and intelligent alert notification function on the web UI

To enhance user experience and optimize remote monitoring capabilities, the system integrates a visual message feedback function based on AI decisions:

- Status feedback synchronization: Whenever a control command is executed (whether manually, automatically, or via AI), the actual physical state of the relay at the slave node is packaged back into a JSON string and uploaded to the web UI to change the color of the device icon (green: ON, gray: OFF). This confirms that the hardware device has successfully triggered in the real world.
- Dynamic AI alert banner: The web UI automatically analyzes the `ai_decision` variable from the database to generate corresponding alert message boxes:
 - WARNING state: Automatically displays a prominent yellow banner stating: "AI WARNING: Soil is drying out in the next 30 minutes! Prepare to irrigate."
 - PUMP_ON state: Automatically switches to a dangerous red banner stating: "AI SYSTEM: Soil is too dry! Activating proactive water pump."
 - SAFE state: Automatically hides all message boxes to keep the user interface clean and organized.

4.6. Safety and system protection functions

- Communication loss fail safe state: To prevent the risk of water resource waste or plant pot waterlogging, a fail safe routine is programmed directly onto the slave nodes. If a slave fails to receive coordination signals or data polling packets from the gateway master via the LoRa network for three consecutive cycles (equivalent

to 15 minutes), the microcontroller automatically cuts off power to the relays, reverting all water pumps and cooling fans to a SAFE state (OFF).

- Watchdog timer integration: Both the gateway and the nodes integrate the `esp_task_wdt` (watchdog timer) system library. If the software freezes or hangs up during code execution, the watchdog automatically forces a hardware reset on the ESP32 to restore operations.
- LoRa hardware self recovery: The `Rst_LORA()` function utilizes an error tracking counter (`retryCount`). If the LoRa module suffers a hardware lock up or interface crash, the ESP32 actively resets the module. In cases of prolonged link disconnections, the firmware proactively re invokes `LoRa.begin()` to rebuild the wireless channel.

Table 4.1. Functional summary by system components

Component	Main function	Execution technology / protocol
Sensor nodes	- Measure local microclimate parameters and soil moisture. - Execute relay opening/closing operations. - Run the automated fail safe routine upon signal loss.	- DHT11, BH1750, and capacitive soil moisture sensors. - GPIO digital control libraries. - RF LoRa SX1278 (433 MHz) transceiver module.
Gateway master	- Coordinate the data request response polling schedule. - Run intermittent irrigation and deadband control algorithms (AUTO mode). - Handle two way data synchronization between the hardware layer and the cloud. - Manage the dynamic network polling list based on the is_active flag.	- Wireless LoRa protocol combined with Wi-Fi connectivity. - Firebase ESP32 client library. - ESPAsyncWebServer library (handling /set_mode and /delete_node APIs).
Firebase cloud	- Provide intermediate real time database functions. - Store global configuration flags, status properties, and control modes.	- Firebase realtime database (JSON tree structure).
Web dashboard	- Display real time sensor measurements and data visualization charts. - Provide a dropdown menu for 3 mode control switching. - Display a dynamic banner system for intelligent AI alerts. - Offer an intuitive user interface to add or delete nodes.	- HTML5, CSS3 Grid/Flexbox. - Vanilla JavaScript (AJAX XMLHttpRequest) and Chart.js.
Edge AI server	- Handle time series data logging and automated data deduplication. - Run a multivariate LSTM deep learning model for soil moisture forecasting. - Convert prediction results into explicit classification control commands (ai_decision).	- Python language (Pandas, Joblib). - LSTM deep learning architecture (TensorFlow / Keras). - Firebase admin SDK library.

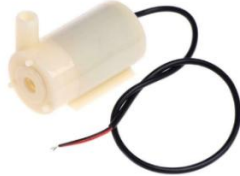
CHAPTER 5: DETAILED DESIGN

5.1. Hardware design

5.1.1. Component list

Table 5.1. Component list of the system.

No	Component name	Quantity
1. Processing and communication section		
1	30 Pin ESP32 Wi-Fi + Bluetooth CP2102-M development kit	3
2	LoRa SX1278 (Ra-02 433 MHz) module	3
3	2.4 GHz Wi-Fi antenna with SMA to IPEX3 adapter cable	3
2. Sensor section		
4	Capacitive soil moisture sensor	2
5	DHT11 temperature and humidity sensor module	2
6	GY 30 BH1750 light intensity sensor module	2
3. Peripheral actuators section (water pump and cooling fan)		
7	2 channel 5VDC relay module (with optocoupler isolation)	2
8	5VDC mini submersible water pump	2
9	Flexible vinyl tubing (water pipe)	2
10	5VDC cooling fan	2
4. Power supply and accessories section		
11	5V - 2A power adapter	2
12	Female DC barrel jack (5.5 x 2.1 mm)	2
13	Jumper wires, breadboards, and connecting wires	x

1. 30 pin ESP32 Wi-Fi + Bluetooth CP2102-M development kit 	2. LoRa SX1278 (Ra-02 433 MHz) module 
3. 2.4 GHz Wi-Fi antenna with SMA to IPEX3 adapter cable 	4. Capacitive soil moisture sensor 
5. DHT11 temperature and humidity sensor module 	6. GY-30 BH1750 light intensity sensor module 
7. 2-channel 5VDC relay module (with optocoupler isolation) 	8. 5VDC mini submersible water pump 
9. Flexible vinyl tubing (water pipe) 	10. 5VDC cooling fan 

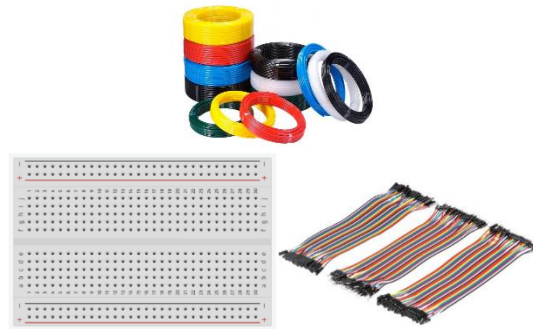
11. 5V - 2A power adapter



12. Female DC barrel jack (5.5 x 2.1 mm)



13. Jumper wires, breadboards, and connecting wires



5.1.2. Schematic diagrams

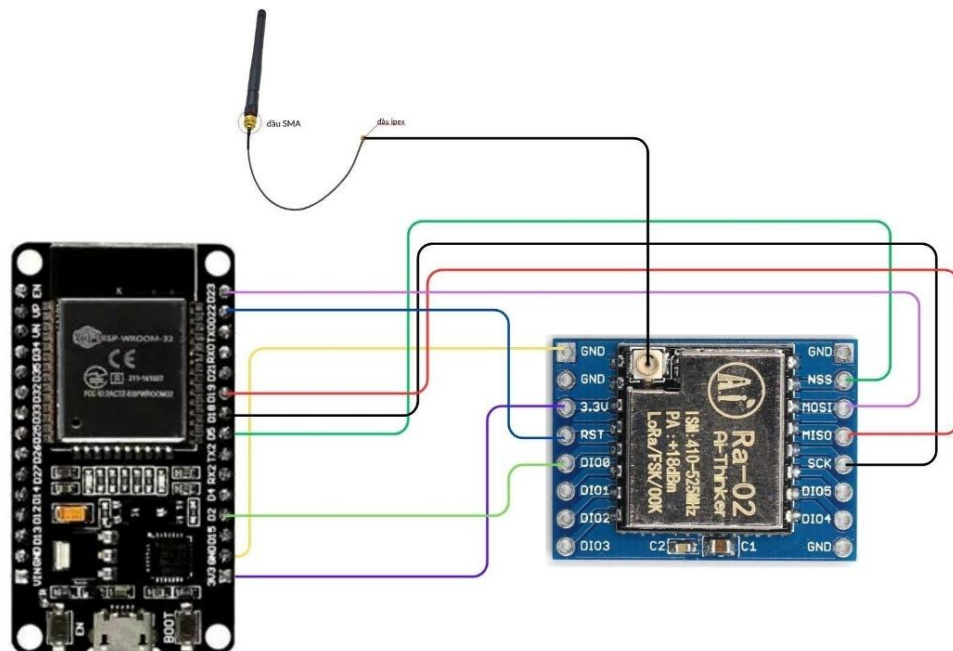


Figure 5.1. Gateway (master) schematic diagram.

Table 5.2. Pin mapping table of gateway (master) components.

LoRa Ra-02 pin	ESP32 pin (GPIO)	Notes
3.3V	3.3V	Power supply input
GND	GND	Ground reference
NSS	D5	Chip Select line (SPI)
SCK	D18	Serial Clock line (SPI)
MISO	D19	Master input slave output data line (SPI)
MOSI	D23	Master output slave input data line (SPI)
RST	D22	Module hardware reset pin
DIO0	D2	Hardware interrupt pin

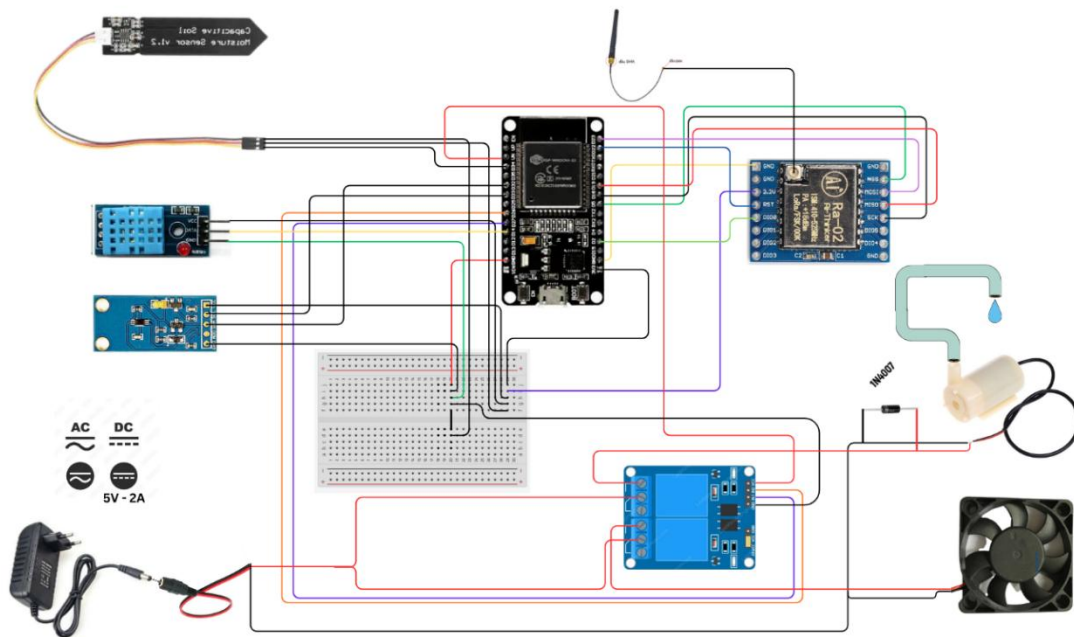


Figure 5.2. Sensor node schematic diagram.

Table 5.3. Pin mapping table of sensor node components part 1.

(sensing and communication).

Component	Component pin	ESP32 pin	Notes
LoRa Ra-02	SPI Pins	Same as Master configuration	Equipped with external antenna
DHT11 sensor	DATA	GPIO 14 (D14)	Powered by 3.3V
Soil moisture sensor	AOUT	GPIO 34 (D34)	Powered by 3.3V
BH1750 sensor	SDA	GPIO 32 (D32)	Powered by 3.3V
	SCL	GPIO 33 (D33)	Powered by 3.3V
Relay module	Signal (IN1)	GPIO 27 (D27)	Controls water pump
	Signal (IN2)	GPIO 26 (D26)	Controls cooling fan
	VCC	VIN	
	GND	GND	

Table 5.4. Pin mapping table of sensor node components part 2.

(actuators and power routing)

Component	Component pin	Water pump connection	Cooling fan connection	Relay connection	Notes
Relay module	NO (Channel 1)	Positive (+) wire			
	NO (Channel 2)		Positive (+) wire		
Jack DC	Negative (-)	Negative (-) wire	Negative (-) wire		
	Positive (+)			COM (Channel 1)	Feeds water pump loop
					COM (Channel 2)
The 5V - 2A power adapter is coupled directly into the female DC barrel jack (5.5x2.1mm).					

5.2. Algorithm and processing flow design

The system operates based on a combination of an asynchronous programming architecture on the web server, a sequential data polling loop on the gateway, and an event driven architecture handling closed loop feedback commands.

5.2.1. Processing logic at the sensor nodes

The firmware running on the Slave nodes is minimally structured to save energy and optimize channel occupation time on the wireless LoRa network, operating under a targeted request response pattern:

1. System Initialization: Configure the GPIO pins, initialize the I2C interface for the BH1750 light sensor, set up the digital interface for the DHT11, configure the Analog to Digital Converter (ADC) for the soil moisture sensor, and initialize the LoRa SX1278 module to operate on the 433 MHz frequency band.
2. Sensor data collection: Periodically read the raw analog voltage value from the soil moisture sensor via the ESP32 ADC channel. To convert this raw reading into a real numbered soil moisture percentage (0% - 100%), the algorithm executes a linear interpolation function (mapping) based on two empirical calibration marks: the maximum value recorded when the sensor is placed entirely in dry air ($ADC_{dry} \approx 3120$, corresponding to 0% moisture) and the minimum value recorded when the sensor probe is fully submerged in water ($ADC_{water} \approx 1450$, corresponding to 100% moisture) according to the following mathematical equation:

$$\text{Moisture (\%)} = \frac{ADC_{dry} - ADC_{raw}}{ADC_{dry} - ADC_{water}} \times 100\%$$

The output value is subsequently bounded within the [0, 100] range using the constrain() function to eliminate edge errors caused by instantaneous noise variations.

3. Polling and verification: The Slave remains in Rx mode awaiting master queries. If the incoming target identity matches the local Node ID (e.g., S1 or S2), it transmits the packaged telemetry back; otherwise, the packet is discarded.
4. Downlink and fail safe execution: The node intercepts flat string actuator commands structured as [Device_ID, State]. To prevent continuous flooding or waterlogging due to communication dropouts, a fail safe check is integrated: if no valid packet arrives from the Master for three consecutive polling cycles (15 minutes), all relays are automatically forced to a safe deactivated state (OFF).

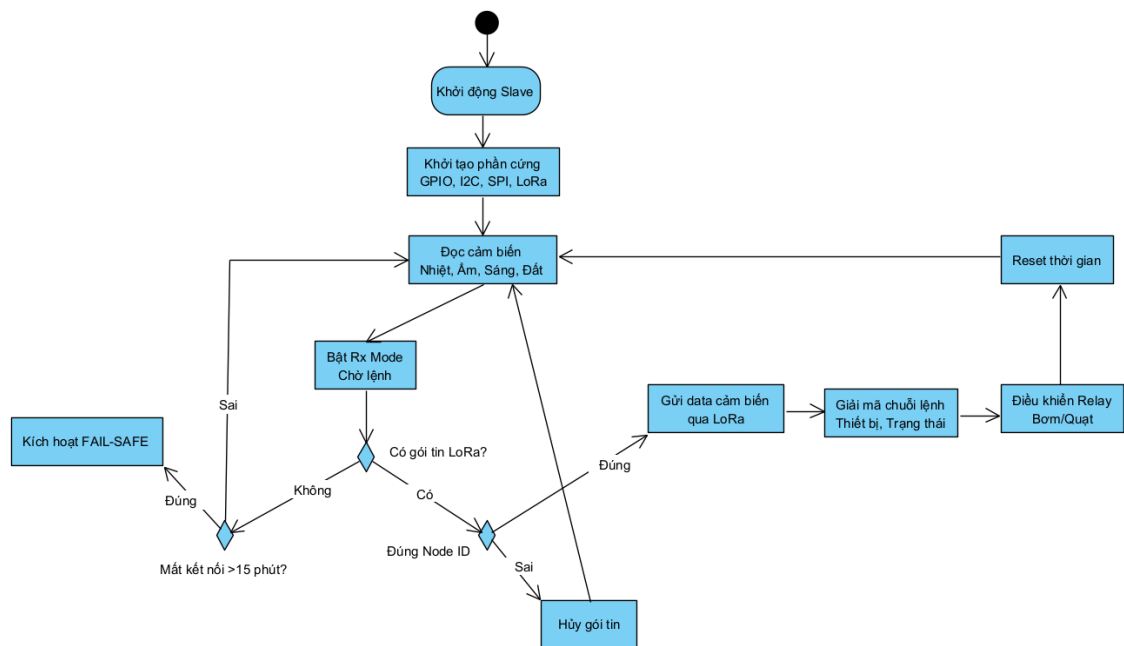


Figure 5.3. Processing logic at the sensor nodes.

5.2.2. Processing logic at the central coordination station (gateway master)

The gateway master functions as the central bridge, coordinating the wireless network polling loop and synchronizing operational states with the cloud through a five step execution sequence:

1. Initialization and network connection: Activates the LoRa SX1278 transceiver, establishes local Wi-Fi routing, and authenticates the secure client token with the firebase realtime database.
2. Dynamic configuration verification: Queries the firebase `is_active` flag for each node prior to a new loop. If true, the node is appended to the active polling sequence; if false (deleted/disabled via the Web UI), the master bypasses it to eliminate empty timeout latency.
3. Collision avoidance polling: Sequentially broadcasts targeted data requests to individual nodes (e.g., Node 1, then Node 2). The Master waits for a response window before switching nodes, ensuring only one device transmits at a time to eliminate shared channel RF interference.
4. Data synchronization: Decodes the incoming LoRa packet payload, extracts the raw sensor parameters, and synchronizes the parsed telemetry up to firebase using an incremental indexing pointer (`latestID`).
5. Three mode decision tree processing: Evaluates the node's `control_mode` variable fetched from firebase to execute the corresponding automated or manual scenario:
 - If `control_mode == "MANUAL"`: The Master delegates all control priority to the user, remaining in an idle monitoring state while awaiting manual switching commands from the Web interface.
 - If `control_mode == "AUTO"`: The Master automatically executes localized rule based control algorithms. For the water pump, it runs the intermittent irrigation routine: if the real time soil moisture drops below 55% and the pump

- If `control_mode == "AI"`: The Master bypasses current real time sensor measurements and checks the value of the intelligent predictive variable `ai_decision`. If `ai_decision == "PUMP_ON"` and the physical water pump is currently off, the Master transmits a LoRa command to proactively trigger irrigation. If `ai_decision == "SAFE"` and the pump is running, it transmits a deactivation packet to protect water resources.



5.2.3. LoRa packet transmission and reception logic

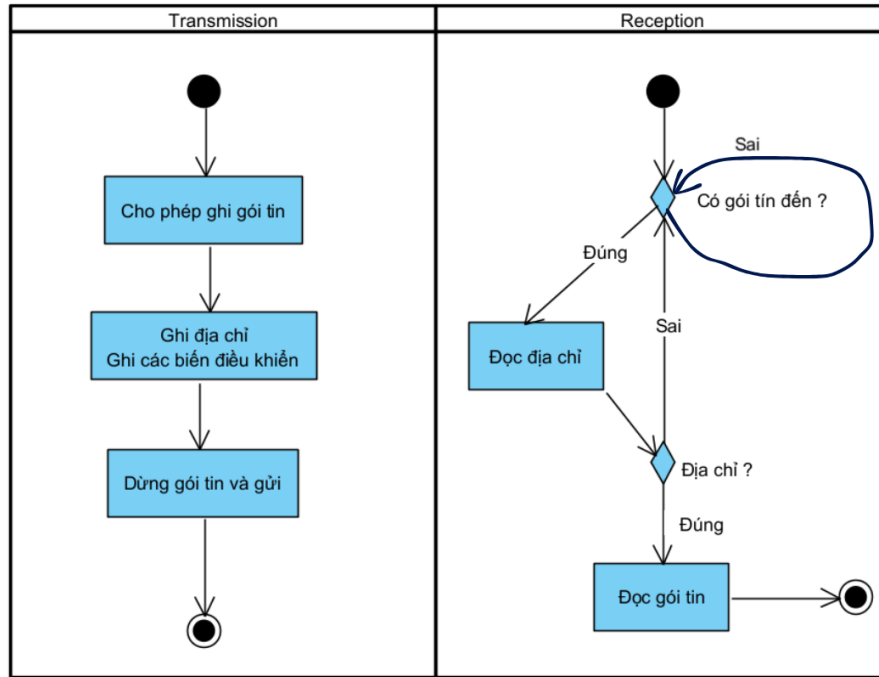


Figure 5.5. Flowchart of the LoRa packet transmission and reception process.

5.2.4. RSSI processing and distance estimation algorithm at the master

Immediately after successfully receiving a LoRa packet using the `LoRa.parsePacket()` command, the ESP32 microcontroller on the gateway master station calls the `LoRa.packetRssi()` function to extract the actual received signal strength indicator (RSSI) value from the hardware structure of the SX1278 chip (for example, -54 dBm).

To convert this RSSI value into a geometric distance, the master applies the log distance path loss model established theoretically in Section 3.3. The system sets fixed configuration parameters, including the signal strength at a 1 meter reference distance $RSSI_0 = -40$ dBm and the path loss exponent $n = 2.5$.

It should be noted that the constant values $RSSI_0 = -40$ dBm and $n = 2.5$ are empirical data determined through a separate series of experimental measurements during the

project execution, matching the hardware characteristics of the Ra-02 module using a spring antenna under the specific geometric constraints of the project's laboratory environment.

Because field radio waves in the 433 MHz band always experience strong non linear effects from multipath reflection, terrain diffraction caused by iron frames, and signal absorption from canopy moisture density, the mathematical result extracted from the distance variable only provides a relative estimate of the physical link range. This model does not serve as a precise coordinate positioning algorithm; its core purpose is to provide intuitive validation data so that operators can recognize the far/near connection status of the edge network nodes.

Once successfully calculated, the value of the distance variable is packaged along with the rssi metric to be synchronized to the /Network_Config/Active_Nodes/{Node_ID}/ path on the firebase realtime database. This data stream serves as the foundational structural input for visualizing the real time network status on the user's web dashboard.

5.3. Database design (firebase structure)

The database is organized using a non relational tree architecture (NoSQL JSON tree topology) to achieve optimal retrieval speed with minimal latency, serving both hardware tasks and deep learning algorithms concurrently.

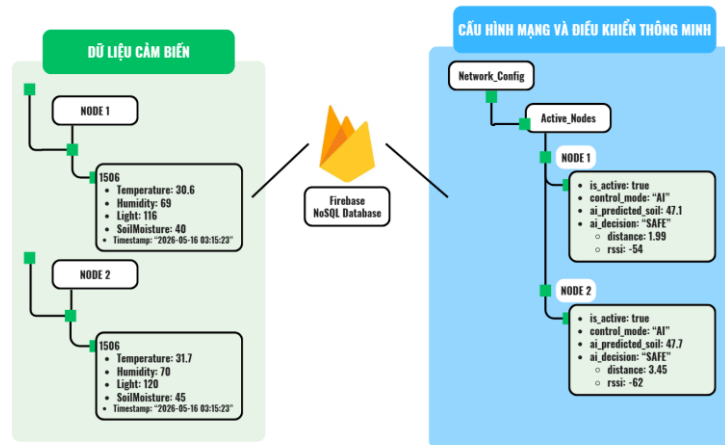


Figure 5.6. Tree architecture storage layout on firebase database.

- Sensor data branch (/Node1, /Node2): Stores the historical time series environmental data. The latestID field acts as an index pointer, enabling the python data logger script and the Web UI to directly retrieve the latest record without scanning the entire data tree, which significantly reduces network bandwidth usage.
- Network configuration and intelligent control branch (/Network_Config/Active_Nodes/): This is the central nerve center for system status synchronization. It integrates all dynamic parameters, including the polling activation flag (is_active), the 3 mode control switch flag (control_mode), the continuous real value predicted by the LSTM model (ai_predicted_soil), and the discrete control classification string (ai_decision).

5.4. User interface design (web dashboard)

The web UI interface is built using standard HTML5, CSS3 Grid/Flexbox, and vanilla JavaScript (Vanilla JS) to ensure fast page load speeds and multi device display compatibility. The interface is modularized into three intuitive functional zones:

- Real time microclimate monitoring zone: Integrates the Chart.js library to visualize ongoing changes in air temperature, air humidity, light intensity, and soil moisture as continuous line charts. These parameters are enclosed within distinct UI cards for each individual node, accompanied by the radio link quality metric (RSSI) and the estimated distance from the node to the gateway.
- Control and dynamic mode switching zone: Inside each Node's card, the system implements a selection dropdown menu (Manual / Auto / AI). When a user changes the selection, a background AJAX XMLHttpRequest process is triggered to transmit the parameter to the /set_mode API on the Master, instantly updating the system configuration without a page reload. Interacting with the toggle

switches to turn the pump and fan on or off is only unlocked when the dropdown is set to MANUAL mode.

- AI message notification zone (dynamic UI box): This functional module uses event listeners to continuously monitor the value of the ai_decision field in the database. If the status changes to WARNING or PUMP_ON, the interface automatically injects dynamic HTML elements to display prominent alert banners with matching color schemes (yellow for warnings, red for emergency activation). This allows operators to intuitively monitor soil risk trends.

5.5. LSTM model

5.5.1. Predict phase – soil moisture forecasting

The forecasting process is executed on the edge server using a time series regression model:

Input configuration: The system uses a sliding window technique to segment input data into a 3D tensor matrix with a shape of (1, 12, 4). This captures 12 consecutive historical timesteps (past 60 minutes) across 4 parallel sensor features:

[Temperature, Humidity, Light, SoilMoisture].

- 60 minute window rationale: With a fixed 5 minute hardware sampling interval, a 12 timestep window captures microclimatic trends (such as solar or thermal gradients during sunrise or cloud cover). This window is long enough to preserve temporal patterns yet short enough to mitigate the vanishing gradient problem and limit edge processor overhead.

Model operational mechanism: A sequential two layer LSTM recurrent neural network (layer 1 with 64 units, layer 2 with 32 units, combined with interleaved 20% dropout layers to prevent overfitting) analyzes the correlation between the soil moisture depletion rate and the changing impacts of ambient temperature and light intensity. The final dense output layer uses a linear activation function for mathematical regression.

- 64 - 32 node configuration rationale: Stacking layers increases abstract feature representation. Layer 1 (64 nodes) extracts cross correlations among the 4 input features, while layer 2 (32 nodes) condenses these temporal characteristics into a hierarchical vector before regression. This structure achieved the lowest mean squared error (MSE) without causing inference latency.
- 0.2 dropout rate rationale: Disconnecting 20% of neurons randomly per epoch eliminates co adaptation and prevents overfitting against field sensor noise. Empirical grid testing across thresholds [0.1, 0.2, 0.3, 0.5] indicated that 0.1 permitted validation loss divergence, while 0.3 and 0.5 degraded convergence rates and raised the baseline MSE. The 0.2 rate minimized the mathematical training to validation loss gap ($\Delta = 0.0002$).

Output configuration: Returns a single continuous real numbered value representing the predicted soil moisture 30 minutes into the future ($\hat{y}_{t+6} = \text{Soil_Moisture}_{t+30}$)

- 30 minute horizon rationale: Based on soil physical dynamics and closed loop lag constraints. Because soil moisture shifts slowly and the intermittent irrigation protocol (10s pumping, 3min waiting) requires time for volumetric water diffusion, a 30 minute predictive window creates a reliable "reaction buffer." This allows the edge server to issue proactive downlink commands to the master station, triggering irrigation precisely before the field soil hits critical drought boundaries.

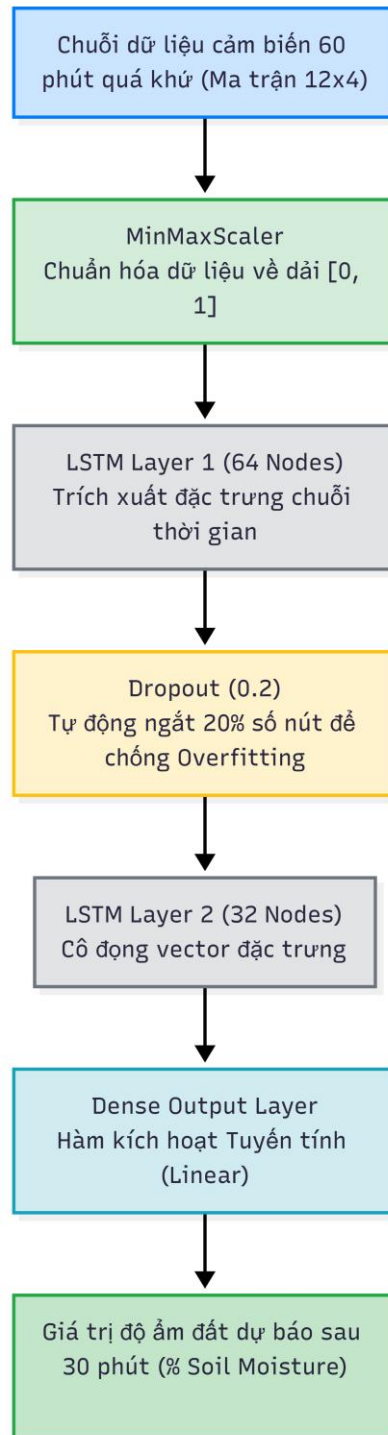


Figure 5.7. Data processing workflow and structural layers of the deep learning LSTM network in the system.

5.5.2. Classify phase – soil status classification and decision making

Upon receiving the predicted real valued number from the LSTM model, the edge server does not transmit the raw number directly to the hardware. Instead, it processes it through a rule based logic classifier to issue accurate control commands based on the following decision mapping matrix:

Table 5.5. Soil status classification and decision making matrix.

Predicted soil moisture value	Soil status classification	Control decision	System feedback behavior
> 40%	Sufficient moisture	SAFE	- Deactivate water pump (OFF). - Hide all alert banners on the Web UI.
30–40%	Slightly dry soil	WARNING	- Maintain water pump in OFF state. - Trigger yellow early warning banner on the Web UI.
< 30%	Dry soil	PUMP_ON	- Trigger red alert banner on the Web UI. - Transmit LoRa command to close the relay and turn on the pump immediately.

The integration of the continuous time series forecasting layer (Predict) and the logical classification layer (Classify) forms an intelligent control algorithm. This enables the system to detect and completely resolve any risks of water deficiency before the physical soil dries out in the field.

CHAPTER 6: SYSTEM IMPLEMENTATION

6.1. Hardware and software deployment environments

This section specifies the precise tools, software versions, and technical infrastructure conditions used to configure the system.

- Hardware Development Environment:
 - o **Compiler:** Arduino IDE (version 2.x) combined with the ESP32 core package (by Espressif).
 - o **Core embedded libraries:** LoRa (by Sandeep Mistry), Firebase ESP32 (by Mobizines), DHT sensor library, and BH1750.



Figure 6.1. Arduino IDE logo.

- AI and data server deployment environment (edge server):
 - o **Operating system:** Windows 11 running on a personal computer acting as the edge server.
 - o **Programming language:** Python (version 3.10+).
 - o **Deep learning and data processing libraries:** TensorFlow/Keras, Pandas, NumPy, Scikit learn, and Joblib.
- Web dashboard operational environment:
 - o **Platform:** Web browsers supporting standard HTML5/CSS3/JavaScript (such as Google Chrome or Microsoft Edge).

- Visualization library: Chart.js embedded directly via a content delivery network (CDN).

6.2. Physical construction and hardware assembly

The physical components are packaged into a functional lab scale mini greenhouse prototype through the following deployment steps:

- Distributed sensor slave nodes deployment:
 - Solder or mechanically connect the sensor modules (DHT11, BH1750, and capacitive soil moisture) and the LoRa Ra-02 module onto the ESP32 development kit in accordance with the predesigned SPI/I2C schematic layouts.
 - Connect the designated digital GPIO control pins to the input channels of the isolated relay module.
 - Technical enclosure assembly: Secure all hardware modules inside a waterproof plastic technical junction box to protect the sensitive microcontrollers from high ambient humidity and direct irrigation water splashes.
- Central coordination gateway master station deployment:
 - Assemble the LoRa SX1278 module using an enhanced spring antenna to maximize signal reception sensitivity and packet delivery rates.
 - Connect the gateway to a stable 5VDC - 2A power adapter to maintain consistent LoRa transmission power and ensure uninterrupted Wi-Fi network routing.



Figure 6.2. Technical components packaged inside the prototype enclosure.

6.3. Cloud firebase database installation and configuration

The intermediate cloud storage infrastructure is provisioned and configured on Google cloud using the following procedure:

- Project initialization: Provision a new database instance located specifically within the Southeast Asia region (Asia Southeast1) to minimize geographic network routing and optimize end to end data transmission latency.
- Security rules configuration: Establish strict database access policies that grant complete read and write privileges to both the gateway master and the backend AI server, while restricting the web UI clients to dynamic event listening channels.
- Service account key extraction: Generate a private authentication credential key exported as a JSON formatted service account file (firebase_key.json). This file establishes a secure, encrypted token connection for the backend Python scripts running on the edge server computer.

6.4. AI process and data logging deployment on the edge server

The core automated intelligence is driven by executing a sequence of specialized python scripts running concurrently as background daemons on the laptop edge server:

- Data cleansing and logging process (`data_logger.py`):
 - o Launch a background script that continuously monitors changes in the `latestID` variable of each active node on the database tree.
 - o The system runs a data validation check against historical indices to filter out redundant static duplicates caused by hardware disconnections or channel noise. This guarantees that the compiled `sensor_data.csv` log maintains high data purity for time series modeling.
- Model training process (`train_model.py`):
 - o Once a statistically sufficient dataset is collected, execute this script to normalize data features using a `MinMaxScaler` and feed the vectors into the stacked two layer (64-32 nodes) LSTM architecture.
 - o The training results are exported as a compiled weights model file (`lstm_soil_model_node1.h5`) alongside its matching scaling configuration parameters (`scaler_node1.gz`).
- Real time AI inference activation (`ai_inference.py`):
 - o Run the real time inference script. This process triggers automatically every 5 minutes to capture a sliding window of the 12 most recent data logs from firebase, perform multivariate predictive calculations, and instantly write the categorized logical command (SAFE, WARNING, or PUMP_ON) back to the cloud database.

6.5. Web dashboard deployment and synchronization

The web user interface communicates interactively with the remote field hardware layers using the following implementation strategy:

- Interface source code embedding: The frontend source code file (`web_dashboard.h`) containing raw HTML5, CSS3, and JavaScript structures is

compiled directly into the flash memory of the ESP32 master as a large string literal to minimize embedded storage allocation overhead.

- Local web server initialization: Utilize the ESPAsyncWebServer library to configure the gateway master to host a local asynchronous HTTP server listening on standard port.
- Two way data flow execution:
 - Uplink: Implement asynchronous JavaScript (AJAX XMLHttpRequest) routines to continuously capture live database streams pushed from firebase, feeding the values directly into Chart.js elements for dynamic real time plotting.
 - Downlink: Establish dedicated API endpoints on the master (such as /set_mode) to intercept mode switching interactions from the client side dropdown menu. This immediately updates the target state on Firebase and broadcasts the corresponding control packet via LoRa to the field actuators.

Note: For a comprehensive overview of the implementation details, the complete source code and step by step guidelines are hosted on my github. The repository link is provided at the end of **Appendix A**.

CHAPTER 7: EXPERIMENTAL RESULTS AND EVALUATION

7.1. Overview of achieved results and practical deployment status

The project successfully implemented an AI integrated smart agricultural monitoring and control system for a mini greenhouse prototype. All components, from the edge hardware layer to the cloud application layer, were verified through continuous operational testing under realistic field conditions.

7.1.1. Hardware deployment

The physical deployment delivered a fully operational setup comprising one central gateway master station and two independent, distributed sensor slave nodes. All electronics are housed inside waterproof technical enclosures to protect the microcontrollers from high humidity and direct water spray.

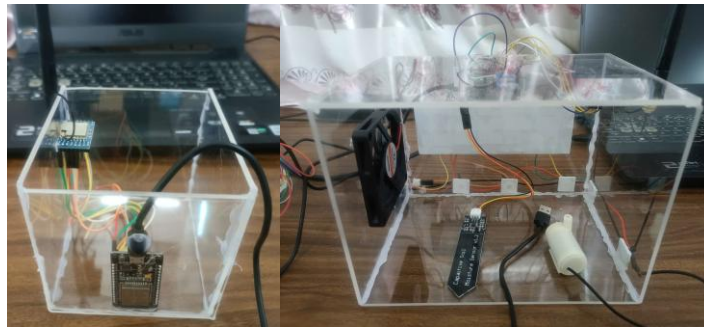


Figure 7.1. Gateway master and a sensor node assembly.



Figure 7.2. Complete hardware setup featuring 1 gateway master node and 2 slave nodes.

7.1.2. LoRa communication network

An internal LoRa network operating on the 433 MHz band has been successfully established and maintains highly stable link connectivity. The sequential request response network scanning mechanism (polling) operates with high precision, completely eliminating radio channel contention or data collision while allowing sensor nodes to be dynamically appended to or removed from the polling cycle via database configuration flags.

7.1.3. Cloud synchronization

TA seamless, real time data connection has been established between the gateway master station and the firebase realtime database cloud infrastructure. Environmental metrics are structured within a NoSQL hierarchical tree configuration, enabling rapid two way data synchronization with optimal network latency.



Figure 7.3. Hierarchical NoSQL JSON tree organization structure for data storage on firebase.

7.1.4. Management interface (web dashboard)

A responsive web UI dashboard has been successfully developed to visually represent microclimate dynamics via real time animated line charts. The dynamic control mode selection menu (Manual / Auto / AI) and the intelligent AI status banner notification box operate in complete synchronization with the remote hardware components.

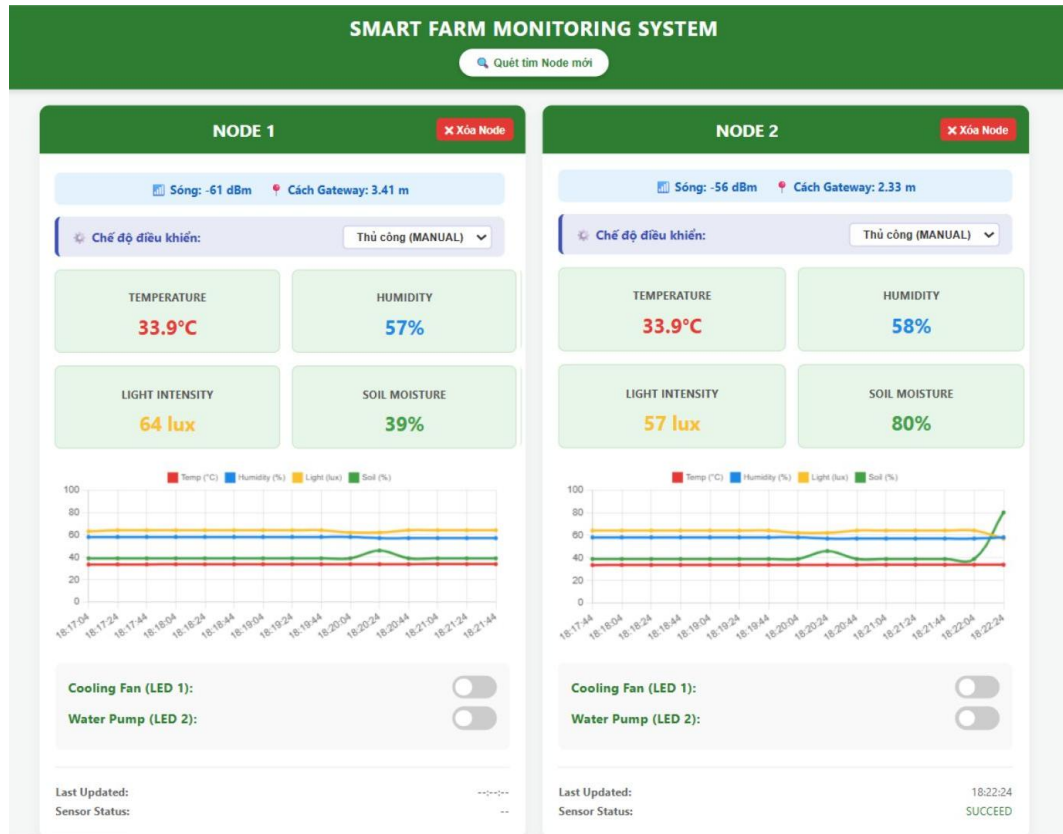


Figure 7.4. Web dashboard user interface managing the distributed sensor nodes.

7.1.5. Edge AI system

The automated backend processing environment is fully operational on the edge server. A dedicated python data logger script continuously extracts new records from Firebase while performing historical validation to filter out redundant static duplicates into a structured .csv log. Concurrently, the trained deep learning LSTM model runs in

an active inference loop processing data windows every 5 minutes to generate predictive analytics and upload proactive control decisions directly onto the cloud database without requiring human intervention.

7.2. Quantitative empirical evaluation of maximum transmission distance

To evaluate the operational limits of the Ra-02 LoRa module (3 dBi spring antenna), the gateway master was fixed in a 5th floor laboratory with concrete wall obstructions while a slave node transmitted 100 packets per checkpoint at increasing distances across campus.

The quantitative parameters gathered include:

- RSSI (received signal strength indicator): The received signal strength power measured at the receiver antenna, expressed in dBm.
- SNR (signal to noise ratio): The ratio of signal power to noise floor power, expressed in dB.
- PDR (packet delivery rate): The percentage of successfully transmitted and acknowledged packets (%), calculated using the following formula:

$$\text{PDR} = \frac{\text{Successfully Received Packets}}{\text{Total Transmitted Packets}} * 100\%$$

Table 7.1. Quantitative empirical data of LoRa signal quality over distance checkpoints.

Distance (m)	Average RSSI (dBm)	Average SNR (dB)	PDR rate (%)
10 m	-52 dBm	+8.5 dB	100%
50 m	-74 dBm	+4.2 dB	100%
100 m	-88 dBm	+1.5 dB	100%
200 m	-98 dBm	-3.5 dB	98.0%
500 m	-114 dBm	-8.2 dB	92.0%
800 m	-126 dBm	-12.5 dB	71.0%
1000 m	-132 dBm	-15.0 dB	42.0%

Note: This experiment was conducted on the university campus grounds to establish baseline bounds, not inside the final practical greenhouse enclosure.

Analysis and discussion of results:

- 10m - 500m range: The link demonstrated high reliability, maintaining a packet delivery rate (PDR) above 92% despite heavy concrete obstructions.
- Beyond 800m: RSSI approached the marginal sensitivity boundary of the SX1278 chip (-148 dBm) and SNR dropped deeply into negative values, causing severe bit errors and a sharp decline in PDR.
- Application fit: Since typical greenhouse sectors operate within a 100m – 300m radius, these findings confirm the architecture fully satisfies wide area coverage requirements without additional signal repeaters.

7.3. Performance evaluation of the AI model (LSTM)

The performance of the sequential two layer LSTM deep learning regression model was evaluated based on the convergence history of the mean squared error (MSE) loss function over a sliding window of 50 training epochs. The dataset used to train the model was extracted from data collected continuously over 11 days of uninterrupted system operation. The raw environmental sensor telemetry saved in the compiled .csv file contains 3,439 distinct historical samples.

7.3.1. Convergence and error margin evaluation

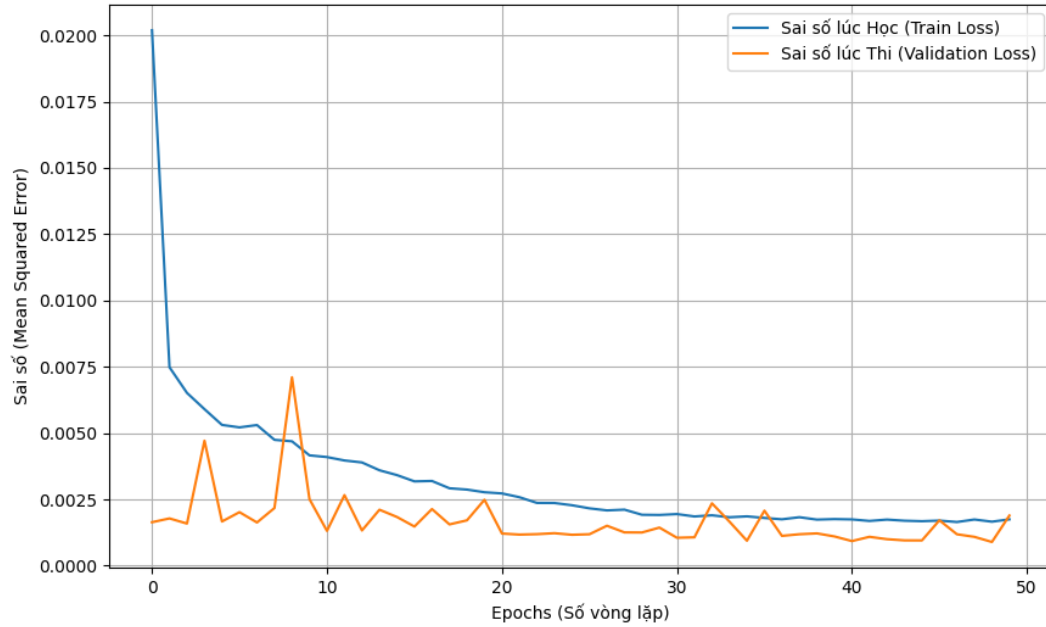


Figure 7.5. Training performance and loss curve evaluation of the LSTM model.

Following the training process, the LSTM deep learning model demonstrated a capacity for global optimization, exhibiting fast convergence and achieving high reliability on the validation dataset. The detailed progression of the mean squared error (MSE) loss function for both the training set (training loss) and testing set (validation loss) is visualized in figure 7.24:

- At epoch 1/50: The model initiates the optimization process with randomly initialized weight matrix parameters. Because it has not yet captured the underlying time series dependencies, the network error remains high, yielding a loss value of 0.0202 and a val_loss value of 0.0016.
- At epoch 25/50: The weight matrices across the functional gates (forget gate, input gate, output gate) begin to approach the global optimum via the Adam optimization algorithm. The system error drops sharply and stabilizes, yielding a loss of 0.0023 and a val_loss of 0.0012.

- At epoch 50/50: The model reaches stable convergence at the baseline of the objective function. The MSE metric on the normalized data scale stops at an optimal level with a loss of 0.0017 and a val_loss of 0.0019.

The mathematical gap between these two loss sets is extremely small ($\Delta = 0.0002$), confirming a well balanced network configuration with zero indications of either overfitting or underfitting.

To evaluate the actual forecasting error of the model in real physical units (% soil moisture), the root mean squared error (RMSE) was calculated. Since all input and output data of the model were previously scaled to the range [0, 1] using the minmaxscaler function, the obtained RMSE value from the network's loss function

($MSE_{normalized} = 0.0017$) initially reflects the normalized data range:

$$RMSE_{normalized} = \sqrt{MSE_{normalized}} = \sqrt{0.0017} \approx 0.04123$$

To convert this error metric back into the real physical measurement scale, the algorithm must perform an inverse calculation (denormalization) by multiplying it by the difference between the maximum and minimum values ($y_{max} - y_{min}$) of the original soil moisture dataset according to the general formula:

$$RSSI_{physical} = RMSE_{normalized} \times (y_{max} - y_{min})$$

Where:

- y_{max} is the upper bound limit of the soil moisture sensor configuration in the field ($y_{max} = 100\%$).
- y_{min} is the lower bound limit of the soil moisture sensor configuration in the field ($y_{min} = 0\%$).

Substituting the experimental quantitative values into the denormalization equation yields the actual physical error of the AI model:

$$RMSE_{physical} = 0.04123 \times (100\% - 0\%) = 4.123\% \approx 4.12\%$$

This quantitative calculation proves that the real world forecasting error of the AI model fluctuates at only 4.12% across the entire physical scale of the sensor. This represents a very low error rate, confirming that the LSTM recurrent network possesses exceptionally high accuracy and reliability in modeling the complex non linear dynamics of soil moisture inside the greenhouse, establishing a solid foundation for the control block to make safe, proactive irrigation decisions.

7.3.2. Real time inference testing results

When deploying the AI engine into live operations via the continuous `ai_inference.py` script, the system executes real time inference cycles and delivers highly consistent results:

- Under stable environmental conditions during dawn, the data curves remain flat. The AI performs inference and outputs a 30 minute ahead soil moisture forecast holding steady between 47.1% and 47.8%. The logic engine returns a SAFE state decision, overwriting the cloud variable to lock the water pump valve and conserve water resources.
- To perform a data shock test, the soil moisture sensor probe was pulled completely out of the soil into dry air. The soil moisture value dropped abruptly. The AI model immediately identified the drought risk trend and switched the `ai_decision` variable to `PUMP_ON`. Consequently, the gateway master instantly fetched the command from the cloud and successfully transmitted a LoRa signal to close the relay, activating the physical water pump in the field.

In addition, performance metrics regarding power consumption, latency, and system scalability are currently being evaluated.

Note: The figures presented above serve as selected illustrations of the system's current functional modules. For a comprehensive demonstration covering all operational use cases and system scenarios, please refer to **Appendix D**.

CHAPTER 8: LIMITATIONS AND FUTURE DIRECTIONS

8.1. Limitations of the current system

Although the system has achieved its initial design objectives and successfully operated across three modes, practical experimental deployment has revealed several technical limitations that must be objectively acknowledged:

1. **Prototype scale constraints:** The current system is limited to a laboratory test bed prototype (mini greenhouse model). The primary focus of this thesis was to validate the viability of the integrated LoRa + Cloud + AI architecture; thus, mechanical durability and long term operational stability have not been tested in a real world greenhouse environment (which features high humidity, extensive netting, and structural iron frames that cause severe signal attenuation).
2. **AI training dataset scale:** The dataset utilized to train the LSTM model is chronologically constrained, comprising only 11 days of continuous collection with 3,439 clean samples. As a result, the data does not yet account for seasonal weather variations or a diverse range of extreme climate scenarios. The evaluated forecasting error of 4.12% is localized to this specific dataset and is insufficient to guarantee the long term robustness of the model.
3. **Lack of baseline evaluation models:** The thesis has not yet conducted a quantitative performance comparison between the proposed LSTM network and traditional time series forecasting models or alternative machine learning architectures (such as ARIMA, moving average, random forest, or GRU) to mathematically highlight the distinct advantages of the selected solution.
4. **Incomplete system performance evaluation:** The validation process has not yet included precise power consumption measurements of the Slave stations (particularly during active LoRa transmissions). Furthermore, the end to end

transmission latency and the scalability boundaries regarding the maximum number of nodes managed by the gateway master remain unquantified.

5. Sensor accuracy and calibration limits: The DHT11 microclimate sensor exhibits relatively large technical tolerances ($\pm 5\%$ RH and $\pm 2^\circ\text{C}$). Additionally, the capacitive soil moisture sensor has only undergone linear calibration based on two boundary environments (dry air and distilled water) without deep calibration tailored to the specific physical and chemical properties of different agricultural soils or substrates.
6. Network Infrastructure and Edge Server Dependencies: The AI control mode depends entirely on global Wide Area Network (WAN) connectivity via Firebase Cloud. Moreover, the edge server currently runs on a personal laptop rather than a dedicated, standalone low power hardware unit. If a WAN disconnection or an edge server failure occurs, the AI inference pipeline will be interrupted, forcing the system to fall back to the localized AUTO mode.
7. Wireless communication channel security: The current Peer-to-Peer (P2P) communication protocol between the LoRa nodes transmits packets in plain text. It lacks hardware level cryptographic layers (such as the AES-128 standard native to LoRaWAN), creating vulnerabilities to packet sniffing, interception, or spoofing over the wireless medium.

8.2. Future directions

Based on the technical limitations outlined above, the future expansion and optimization roadmap of this project will focus on the following core domains:

8.2.1. Algorithm upgrades and ai model optimization

- Baseline model construction: Train and run parallel evaluations of the LSTM network against ARIMA, GRU, and random forest models to standardize the performance assessment.

- Dataset expansion and dynamic light ingestion: Extend data collection over a multi season timeline and integrate a daily light integral (DLI) cumulative solar energy calculation algorithm. Adding these real time state variables into the LSTM input 3D tensor matrix will optimize proactive forecasting capabilities during sharp microclimate transitions (such as dawn and dusk).

8.2.2. Hardware optimization and field validation

- Edge hardware hardening and packaging: Transition the Edge Server from a personal computer to dedicated, low power independent embedded computing hardware, such as a Raspberry Pi. Replace the DHT11 sensor with higher precision industrial sensor variants (such as the SHT3x series).
- Real world greenhouse deployment: Deploy the hardware topology directly inside commercial agricultural greenhouses to accurately measure the actual RSSI/SNR signal degradation of LoRa waves under the influence of structural steel frameworks and dense crop canopies.
- Power consumption metrics optimization: Implement deep sleep operational cycles (deep sleep) for the distributed Slave stations, conducting systematic power profiling to optimize battery longevity.

8.2.3. Security enhancements and computer vision integration

- Implement the AES-128 encryption algorithm for the LoRa P2P message payloads or upgrade the overall software stack to comply with the standard secure encryption architecture of the LoRaWAN network specification.
- Computer vision integration: Deploy minimalist camera modules (such as the ESP32-CAM) at the distributed Slave stations, running lightweight deep learning models (e.g., YOLO, MobileNet) on the edge server. This will enable real time visual monitoring of crop health indicators (early detection of pests, diseases, or wilting, and leaf area index measurement) in parallel with raw environmental telemetry.

REFERENCES

Vietnamese

1. Trần Văn Lích, Lê Hồng Nam (2020), “Mạng không dây LoRa cho ứng dụng IoT tầm xa”, *Tạp chí Điện tử, Khoa học và Công nghệ*, 12(2), pp. 45-52.

English

2. Ali M., A. Mohammad and A. Al-Khasawneh (2021), “Performance Analysis of Serial Peripheral Interface (SPI) Protocol in Embedded Systems”, *International Journal of Computer Applications*, 178(4), pp. 12-18.
3. Balasubramanian K. and A. Cellatoglu (2020), “Optimization of Inter-Integrated Circuit (I2C) Bus Data Rate for Multi-Sensor IoT Applications”, *IEEE Transactions on Consumer Electronics*, 66(2), pp. 145-152.
4. Bengio Y., P. Simard and P. Frasconi (1994), “Learning long-term dependencies with gradient descent is difficult”, *IEEE Transactions on Neural Networks*, 5(2), pp. 157-166.
5. Centenaro M., L. Vangelista, A. Zanella and M. Zorzi (2016), “Long-range communications in unlicensed bands: the rising stars in the IoT and smart city scenarios”, *IEEE Wireless Communications*, 23(5), pp. 60-67.
6. Gers F. A., J. Schmidhuber and F. Cummins (2000), “Learning to forget: Continual prediction with LSTM”, *Neural Computation*, 12(10), pp. 2451-2471.
7. Gu F. and J. Zhang (2020), “Private LoRa Peer-to-Peer Networks for Rural Smart Farming and Low-cost Precision Irrigation”, *IEEE Internet of Things Journal*, 7(10), pp. 9845-9856.
8. Greff K., R. K. Srivastava, J. Koutník, B. R. Steunebrink and J. Schmidhuber (2017), “LSTM: A search space odyssey”, *IEEE Transactions on Neural Networks and Learning Systems*, 28(10), pp. 2222-2232.
9. Goodfellow I., Y. Bengio and A. Courville (2016), *Deep Learning*, MIT Press, Cambridge.

10. Hochreiter S. and J. Schmidhuber (1997), “Long Short-Term Memory”, *Neural Computation*, 9(8), pp. 1735–1780.
11. Jagannathan J. and R. Priyadarshini (2021), “Smart Greenhouse Monitoring System Based on Embedded Edge Rules and Private Wireless Link”, *Springer Journal of Automated Software Engineering*, 28(3), pp. 112-126.
12. Khanna A. and S. Kaur (2019), “Evolution of Internet of Things (IoT) and its Significant Impact in the Field of Precision Agriculture”, *Biocatalysis and Agricultural Biotechnology*, 19, pp. 101131.
13. Mekala M. S. and P. Viswanathan (2019), “A Survey on Smart Agriculture IoT with Cloud Computing”, *Microprocessors and Microsystems*, 65, pp. 1-12.
14. Ray P. P. (2017), “An introduction to protocol standards and architecture for smart agriculture as an Internet of Things application”, *IEEE Access*, 5, pp. 26493-26503.
15. Sadowski S. and P. Spachos (2018), “Wireless technologies for smart agricultural monitoring: A comparative study”, *IEEE Computers and Electronics in Agriculture*, 151, pp. 202-211.
16. Savary S., A. Ficke and J. N. Aubertot (2022), “Crop health and its global impacts on the sustainability of agricultural production systems”, *Elsevier Agriculture Systems*, 196, pp. 103320.
17. Shi X., Z. Chen, H. Wang, D. Y. Yeung, W. K. Wong and W. C. Woo (2015), “Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting”, *Advances in Neural Information Processing Systems (NeurIPS)*, 28, pp. 802–810.
18. Shinde V. and S. Patil (2022), “Quantitative Analysis of Microclimate Parameter Variations on Crop Yield in Controlled Greenhouse Environments”, *ScienceDirect Environmental Technology*, 43(2), pp. 201-215.
19. Sinha A., P. Kumar and N. P. Rana (2023), *IoT Architecture for Precision Agriculture: System Integration and Network Topologies*, Springer, Cham.

20. Srbinovska M., C. Gavrovski, V. Dimcev, A. Krkoleva and V. Borožan (2015), "Environmental Parameters Monitoring in Precision Agriculture Using Wireless Sensor Networks", *Journal of Cleaner Production*, 88, pp. 297-307.
21. Zhao W., S. Lin and B. Ji (2021), "Soil Moisture Prediction Using LSTM Networks with Exogenous Meteorological Variables", *Computers and Electronics in Agriculture*, 182, pp. 106001.

APPENDICES

APPENDIX A: CORE PROGRAM SOURCE CODE

Note: To optimize the presentation space, this appendix only extracts the core source code segments that illustrate the control algorithms, deep learning processing, and cloud data synchronization pipelines. The complete and functional source code of the entire system is hosted publicly in the open-source GitHub repository attached at the end of this appendix.

A.1. Three mode routing logic (AI, AUTO, MANUAL) at the gateway master

This code segment (C++ language) is extracted from the `Processing_incoming_data()` function within the `gateway_master.ino` file. It parses the `control_mode` flag from Firebase and automatically routes the actuator control loop commands by broadcasting targeted `sendMessage` LoRa packets down to the corresponding Slave node.

```
277 // =====
278 // KỊCH BẢN ĐIỀU KHIỂN THÔNG MINH 3 CHẾ ĐỘ CHO NODE 1
279 // =====
280 String ctrlMode_S1 = "MANUAL"; // Mặc định nếu không đọc được
281 if (Firebase.getString(firebaseData, "/Network_Config/Active_Nodes/Node1/control_mode")) {
282     ctrlMode_S1 = firebaseData.stringData();
283 }
284
285 // --- CHẾ ĐỘ 1: ĐIỀU KHIỂN BẰNG AI ---
286 if (ctrlMode_S1 == "AI") {
287     String aiDecision = "SAFE";
288     if (Firebase.getString(firebaseData, "/Network_Config/Active_Nodes/Node1/ai_decision")) {
289         aiDecision = firebaseData.stringData();
290     }
291
292     // Chỉ bật bơm (lệnh "2,t") nếu AI yêu cầu PUMP_ON và trạng thái bơm hiện tại đang tắt ("0")
293     if (aiDecision == "PUMP_ON" && LED_2_State_str == "0") {
294         Serial.println("[AI MODE] AI yêu cầu PUMP_ON. Tiến hành bật bơm Node 1...");
295         delay(100);
296         sendMessage("2,t", Destination_ESP32_Slave_1, led_Control_Mode);
297     }
298     // Chỉ tắt bơm (lệnh "2,f") nếu AI báo SAFE và trạng thái bơm hiện tại đang bật ("1")
299     else if (aiDecision == "SAFE" && LED_2_State_str == "1") {
300         Serial.println("[AI MODE] AI báo SAFE. Tiến hành tắt bơm Node 1...");
301         delay(100);
302         sendMessage("2,f", Destination_ESP32_Slave_1, led_Control_Mode);
303     }
304 }
```

```

305 // --- CHẾ ĐỘ 2: ĐIỀU KHIỂN TỰ ĐỘNG THÔNG THƯỜNG (BIẾN THƯỜNG) ---
306 else if (ctrlMode_S1 == "AUTO") {
307     // Tự động bật bơm nếu độ ẩm đất < 55% và bơm đang tắt
308     if (SoilMoisture[0] > 0 && SoilMoisture[0] < 55 && LED_2_State_str == "0") {
309         Serial.println("[AUTO MODE] Độ ẩm đất < 55%. Tiến hành bật bơm Node 1...");
310         delay(100);
311         sendMessage("2,t", Destination_ESP32_Slave_1, led_Control_Mode);
312     }
313     // Tự động tắt bơm nếu độ ẩm đất đạt ngưỡng an toàn >= 70% và bơm đang bật
314     else if (SoilMoisture[0] >= 70 && LED_2_State_str == "1") {
315         Serial.println("[AUTO MODE] Đất đã đủ ẩm (>= 70%). Tiến hành tắt bơm Node 1...");
316         delay(100);
317         sendMessage("2,f", Destination_ESP32_Slave_1, led_Control_Mode);
318     }
319
320     // Tự động điều khiển Quạt tản nhiệt (LED 1) theo nhiệt độ không khí
321     if (Temp[0] > 32.0 && LED_1_State_str == "0") {
322         Serial.println("[AUTO MODE] Nhiệt độ > 32°C. Tiến hành bật quạt Node 1...");
323         delay(100);
324         sendMessage("1,t", Destination_ESP32_Slave_1, led_Control_Mode);
325     }
326     else if (Temp[0] < 29.0 && Temp[0] > 0 && LED_1_State_str == "1") {
327         Serial.println("[AUTO MODE] Nhiệt độ < 29°C. Tiến hành tắt quạt Node 1...");
328         delay(100);
329         sendMessage("1,f", Destination_ESP32_Slave_1, led_Control_Mode);
330     }
331 }
332 // --- CHẾ ĐỘ 3: MANUAL ---
333 // Nếu ctrlMode là "MANUAL", Master giữ nguyên trạng thái để người dùng tự bật/tắt bằng tay từ web
334 }

```

A.2. Data collection and interpolation algorithm at the sensor station (slave)

This code segment (C++ language) is extracted from the main loop() function within the node_slave.ino file. It demonstrates the structural workflow for reading microclimate parameters alongside the empirical mathematical interpolation method used to map the raw capacitive sensor ADC voltage into an absolute soil moisture percentage.

```

254 unsigned long currentMillis_UpdateDHT11 = millis();
255
256 if (currentMillis_UpdateDHT11 - previousMillis_UpdateDHT11 >= interval_UpdateDHT11) {
257     previousMillis_UpdateDHT11 = currentMillis_UpdateDHT11;
258
259     // Đọc cảm biến vi khí hậu (Nhiệt độ, Độ ẩm, Ánh sáng)
260     Humd = dht11.readHumidity();
261     Temp = dht11.readTemperature();
262
263     if (isnan(Humd) || isnan(Temp)) {
264         Humd = 0;
265         Temp = 0.00;
266         send_Status_Read_DHT11 = "f";
267         Serial.println(F("Failed to read from DHT sensor!"));
268         //return;
269     } else {
270         send_Status_Read_DHT11 = "s";
271     }
272     Light = lightMeter.readLightLevel();
273     // Đọc giá trị độ ẩm đất (quy đổi về phần trăm)
274     // Quy trình nội suy giá trị độ ẩm đất (Soil Moisture Calibration)
275     // Giá trị thực nghiệm ADC: 4095 (Mốc khô tuyệt đối) -> 0 (Mốc ngập nước)
276     // Chuyển đổi thành dải phần trăm: 0% -> 100%
277     SoilMoisture = map(analogRead(SoilMoisture_Pin), 0, 4095, 100, 0);
278 }

```

A.3. Asynchronous event listening logic on the web dashboard

This JavaScript code segment is extracted from the web_dashboard.h asset file. It implements a data subscription stream to fetch real-time state changes smoothly without full page reloads and drives the dynamic UI alert banner system using structural responses processed by the remote AI model.

```

357 if (!window.EventSource) {
358     var source = new EventSource('/events');
359
360     source.addEventListener('open', function(e) { console.log("Events Connected"); }, false);
361     source.addEventListener('error', function(e) {
362         if (e.target.readyState != EventSource.OPEN) { console.log("Events Disconnected"); }
363     }, false);
364
365     source.addEventListener('allDataJSON', function(e) {
366         var today = new Date();
367         var time = leading_zeros(today.getHours()) + ":" + leading_zeros(today.getMinutes()) + ":" + leading_zeros(today.getSeconds());
368
369         count_to_Show_Info_no_Data_is_coming = 0;
370         if (document.getElementById("Show_Info").innerHTML == "Mất kết nối với Master ESP32...") {
371             document.getElementById("Show_Info").innerHTML = "";
372         }
373
374         var obj = JSON.parse(e.data);
375
376         if (obj.ID_Slave == "S1") {
377             // Hiển thị RSSI và Khoảng cách cho Node 1
378             document.getElementById("rssi_Slave_1").innerHTML = obj.RSSI || "--";
379             document.getElementById("dist_Slave_1").innerHTML = obj.Distance ? obj.Distance.toFixed(2) : "--";
380
381             document.getElementById("status_read_DHT11_Slave_1").innerHTML = obj.StatusReadDHT11;
382             document.getElementById("status_read_DHT11_Slave_1").style.color = (obj.StatusReadDHT11 == "FAILED") ? "red" : "#2E7D32";
383         }
384     });
385 }

```

```

384 | document.getElementById("humd_Slave_1").innerHTML = obj.Humd;
385 | document.getElementById("temp_Slave_1").innerHTML = obj.Temp.toFixed(1);
386 | document.getElementById("light_Slave_1").innerHTML = obj.Light;
387 | document.getElementById("soil_Slave_1").innerHTML = obj.SoilMoisture;
388 |
389 | document.getElementById("togLED_1_Slave_1").checked = obj.LED1;
390 | document.getElementById("togLED_2_Slave_1").checked = obj.LED2;
391 |
392 | document.getElementById("LTRD_Slave_1").innerHTML = time;
393 | updateChart(charts1, time, obj.Temp, obj.Humd, obj.Light, obj.SoilMoisture);
394 |
395 | // GỌI HÀM CẬP NHẬT CẢNH BÁO AI CHO NODE 1
396 | updateAIAAlertBox("Slave_1", obj.AIDecision);
397 | }
--- |

539 | // Hàm cập nhật trạng thái cảnh báo AI lên màn hình
540 | function updateAIAAlertBox(slaveId, decision) {
541 |     var alertDiv = document.getElementById("ai_alert_" + slaveId);
542 |     if (!alertDiv) return;
543 |
544 |     if (decision === "WARNING") {
545 |         alertDiv.innerHTML = "⚠️ CẢNH BÁO AI: Đất sắp khô trong 30 phút tới! Chuẩn bị tưới.";
546 |         alertDiv.className = "ai-alert alert-warning";
547 |         alertDiv.style.display = "block";
548 |     } else if (decision === "PUMP_ON") {
549 |         alertDiv.innerHTML = "🔥 HỆ THỐNG AI: Đất quá khô! Đang kích hoạt máy bơm chủ động.";
550 |         alertDiv.className = "ai-alert alert-danger";
551 |         alertDiv.style.display = "block";
552 |     } else {
553 |         // Nếu trạng thái là SAFE hoặc trống thì ẩn banner đi
554 |         alertDiv.style.display = "none";
555 |     }
556 | }

```

A.4. Real time inference script on the edge AI server

This Python script segment is extracted from the `ai_inference.py` script file. It represents the infinite execution loop (the AI core worker process) handling sliding window data ingestion, automated LSTM multivariate regression model execution, denormalization mapping, and logical command updates back to the cloud.

```

64 # 3. VÒNG LẶP TUẦN TRA ĐA ĐIỂM
65 print("\nAI ĐÃ VÀO CA TRỰC! Đang theo dõi các Node...")
66
67 while True:
68     for node in ACTIVE_NODES:
69         try:
70             # 1. Lấy dữ liệu 60 phút qua của từng Node
71             recent_data = get_latest_12_records(node)
72
73             if recent_data is not None:
74                 # 2. Tiền xử lý (Lúc này recent_data đã là DataFrame chuẩn)
75                 scaled_input = scaler.transform(recent_data)
76
77                 # Đóng gói thành ma trận 3D (1, 12, 4)
78                 X_input = scaled_input.reshape(1, 12, 4)
79
80                 # 3. AI Dự đoán
81                 scaled_prediction = model.predict(X_input, verbose=0)[0][0]
82
83                 # 4. Dịch ngược kết quả
84                 dummy_array = np.zeros((1, 4))
85                 dummy_array[0, 3] = scaled_prediction
86                 real_prediction = scaler.inverse_transform(dummy_array)[0, 3]
87                 real_prediction = round(real_prediction, 1)
88
89                 # 5. Phân loại Logic
90                 decision = "SAFE"
91                 if real_prediction > 40:
92                     decision = "SAFE"
93                 elif 30 <= real_prediction <= 40:
94                     decision = "WARNING"
95                 elif real_prediction < 30:
96                     decision = "PUMP_ON"
97
98                 print(f"[{datetime.now().strftime('%H:%M:%S')}] {node} | DỰ BÁO 30P TỚI: {real_prediction}% -> LỆNH: {decision}")
99
100                 # 6. Gửi lệnh lên Firebase
101                 db.reference(f'/Network_Config/Active_Nodes/{node}/ai_predicted_soil').set(float(real_prediction))
102                 db.reference(f'/Network_Config/Active_Nodes/{node}/ai_decision').set(decision)
103                 db.reference(f'/Network_Config/Active_Nodes/{node}/control_mode').set("AI")
104
105             else:
106                 print(f"[{datetime.now().strftime('%H:%M:%S')}] {node}: Chưa đủ 12 bản ghi hợp lệ để dự đoán.")
107
108         except Exception as e:
109             print(f"Lỗi tại {node}: {e}")
110
111     print("-" * 50)
112     time.sleep(300)

```

System repository access: The complete, unedited source code files of the system (including C++ firmware for the ESP32 boards, Python AI model configurations, frontend web dashboard scripts, and authentic CSV experimental datasets) are securely packaged and publicly hosted on GitHub at:

[NguyenGiap25072004/Smart-Agriculture-Monitoring-and-Control-System-using-LoRa-network](https://github.com/NguyenGiap25072004/Smart-Agriculture-Monitoring-and-Control-System-using-LoRa-network)

APPENDIX B: FIREBASE SECURITY RULES

To enforce strict system information security and eliminate unauthorized external read/write modifications to the NoSQL JSON tree topology of the wireless sensor network, the access rules under the Rules configuration tab of the Firebase Realtime Database are explicitly defined as follows:

```
{
  "rules": {
    ".read": "auth != null",
    ".write": "auth != null",
    "Network_Config": {
      "Active_Nodes": {
        "$node_id": {
          ".validate": "newData.hasChildren(['is_active', 'control_mode', 'ai_decision'])"
        }
      }
    }
  }
}
```

APPENDIX C: SAMPLE TIME-SERIES EMPIRICAL DATASET

The layout below illustrates the data structure of the first 10 data rows extracted from the cleansed system log file (sensor_data.csv), chosen out of the total 3,439 consecutive samples captured across the 11 day field monitoring validation period:

	A	B	C	D	E	F
1	Timestamp	Node_ID	Temperature	Humidity	Light	SoilMoisture
2	5/5/2026 0:00	Node1	24.72	87.05	5	60.51
3	5/5/2026 0:00	Node2	24.72	87.05	5	60.51
4	5/5/2026 0:05	Node1	24.82	87.41	5	60.25
5	5/5/2026 0:05	Node2	24.82	87.41	5	60.25
6	5/5/2026 0:10	Node1	24.72	87.42	5	60.34
7	5/5/2026 0:10	Node2	24.72	87.42	5	60.34
8	5/5/2026 0:15	Node1	24.66	88	0	60.07
9	5/5/2026 0:15	Node2	24.66	88	0	60.07
10	5/5/2026 0:20	Node1	24.68	87.49	5	60.4

Figure C.1. Sample structure of raw historical entries stored within the system CSV log file.

APPENDIX D: EXPERIMENTAL OPERATION DEMO VIDEO

The video recording documents the complete operational integration and verification workflows under multiple simulated control scenarios of the mini-greenhouse system within the laboratory infrastructure. It can be accessed directly via the following repository path:

https://drive.google.com/drive/folders/1Zeh0ALfFz_A6chj0026_r26CTAWUVcfP?usp=drive_link